



德卡移动开发包接口文档

***For Java Developers v1.5.6**

德卡移动开发包接口文档
*For Java Developers v1.5.6

一、首页

- 1、概述
- 2、文档修订记录
- 3、 workflow

二、项目导入

Step 1

Step 2

三、智能终端

- 1、智能终端端口打开示例

四、读卡器上下电操作

- 1、Z90N, M200, P18F上下电操作
- 2、P10平板上下电操作

五、调用示例

- 1、设置SDK调用系统语言环境
- 2、端口操作

打开

2.1 USB

2.2 串口

2.3 蓝牙

2.4串口路径

关闭

- 3、卡型操作

- 1、接触式CPU卡

- 1.1 接触式CPU卡简易流程图
 - 1.2 示例代码
- 2、非接CPU卡
 - 2.1 Type A卡简易流程图
 - 2.2 示例代码
- 3、M1卡
 - 3.1 M1卡简易流程图
 - 3.2 示例代码
- 4、磁条卡
 - 4.1 同步方式
 - 4.2 异步方式
- 5、身份证
 - 5.1 读取身份证信息
 - 5.2 读取身份证原始数据信息并进行解析
- 6、二维码
 - 6.1 二维码扫描
- 7、4442卡
 - 7.1 检测4442卡
 - 7.2 验证4442卡密码
 - 7.3 读取4442卡密码
 - 7.4 读取4442卡密码计数
 - 7.5 修改4442卡密码
 - 7.6 读4442卡保护位
 - 7.7 写4442卡保护位
 - 7.8 读4442卡数据
 - 7.9 写4442卡数据

7.10 4442卡下电操作

8、4428卡

8.1 检测4428卡

8.2 验证4428卡密码

8.3 读取4428卡密码

8.4 读取4428卡密码计数

8.5 修改4428卡密码

8.6 读4428卡保护位

8.7 写4428卡保护位

8.8 读4428卡数据

8.9 写4428卡数据

8.10 4428卡下电操作

9、Fm11rf005卡

9.1 读取卡内数据

10、102卡

10.1 检测是否存在102卡

10.2 校验102卡密码

10.3 修改12卡密码

10.4 读102卡密码计数

10.5 读102卡数据

10.6 写102卡数据

10.7 102卡下电操作

10.8 对102卡进行熔丝操作

4、打印模块

1、打印图片

2、打印字符

3、打印一维码

六、常见问题

1、项目配置

Q1:无法正常调用jar包中的类方法?

Q2:java.lang.UnsatisfiedLinkError

Cannot load library ?

Q3:清单文件中已经注册了蓝牙权限，但还是无法使用蓝牙？

Q4:我不知道在清单文件注册哪些？

Q5:libwlt2bmp.so: has text relocations?

2、接口调用

Q1: APDU长度超过255，部分接口返回成功，Apdu部分却为空？

Q2: 我的Android设备串口没有操作权限，能否在打开之前修改文件权限？

七、错误码说明

八、弃用列表

九、接口说明

1、文件列表

1.1BasicOper.java

一、首页

1、概述

此接口文档旨在帮助开发者在Android平台接入读卡器。

注意：接口功能支持包含的部分不一定所有设备都支持，可能一些设备只支持接口功能的某一部分

2、文档修订记录

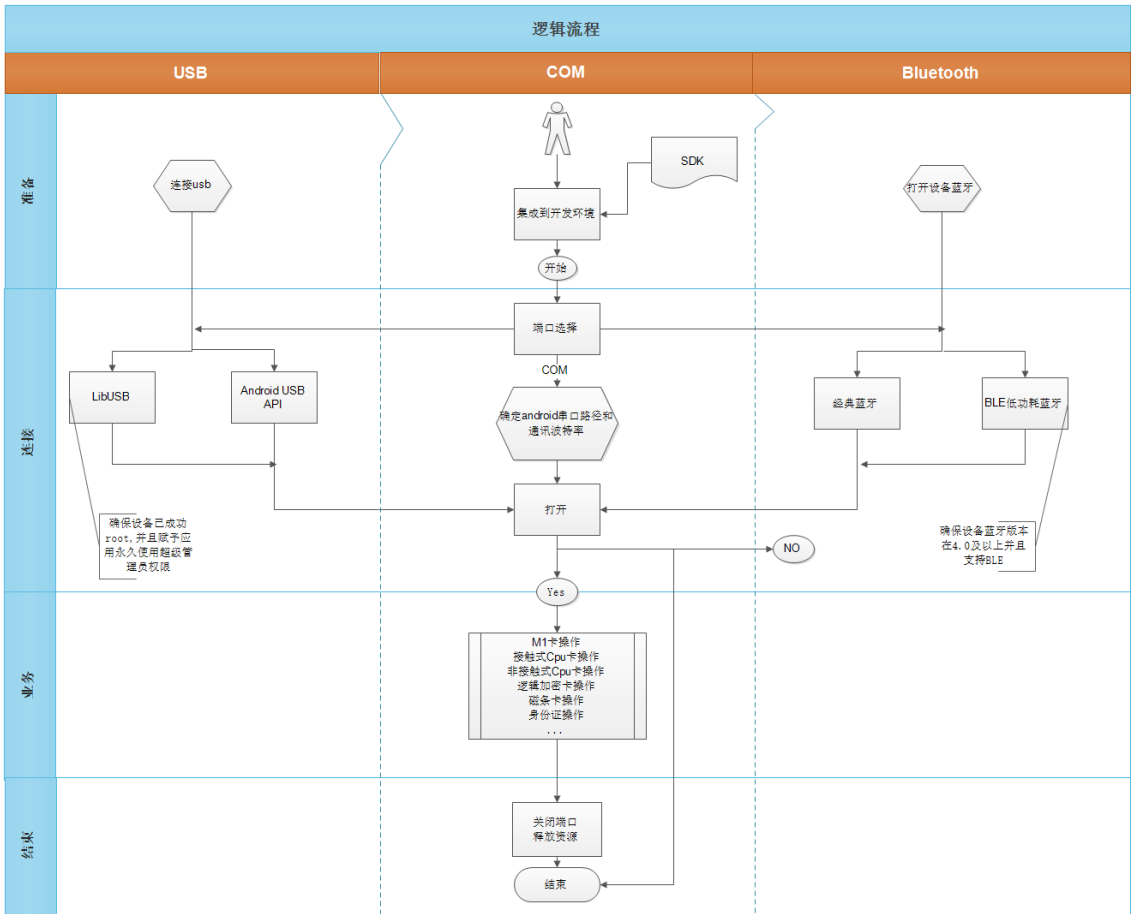
版本号	修改时间	修改内容
<1.1.8	-2017/11/24	初始版本更替
1.1.8	2017/11/28	添加"主页", "项目导入","调用示例","常见问题"
1.1.9	2017/12/01	1,"常见问题"增加apk运行无法找到库的原因之一 2,"蜂鸣器"接口调用提供建议小于50
1.2.0	2017/12/06	1,添加M1卡，接触式CPU卡，非接触式CPU卡(TypeA)基本操作流程 图。
1.2.1	2017/12/07	1,首页添加workflow流程图 2,"常见问题"增加关于串口权限处理的问题和建议。
1.2.2	2018/04/10	1,添加二维码扫描调用示例代码 2,添加液晶显示支持中文方法 dc_dispinfoSrc。
1.2.3	2018/05/17	1,添加打印图片接口示例代码。
1.2.4	2018/05/22	1,添加lcd显示扩展接口说明， 2，添加设置接触式卡座电压接口。
1.2.5	2018/06/26	1,添加磁条卡同步，异步读取接口示例代码。
1.2.6	2018/06/27	1,添加读取身份证模块序列号接口注释。
1.2.7	2018/07/20	1,添加密码键盘通讯接口。
1.2.8	2018/08/09	1,添加读取健康卡信息接口。
1.2.9	2018/09/11	1,添加读取身份证物理ID接口。
1.3.0	2019/01/24	1,添加验证M1卡密钥接口说明，修改示例代码。
1.3.1	2019/04/09	1,添加外接密码键盘接口说明。
1.3.9	2020/01/14	1,优化社保卡业务。

版本号	修改时间	修改内容
1.3.9	2020/03/12	1,增加接口dc_cpuapduresponInt_hex。
1.3.9	2020/05/27	1,增加102卡相关接口。
1.3.9	2020/06/19	1,优化身份证公安部协议读取问题。
1.3.9	2020/08/07	1,增加以下接口：dc_card_fm11rf005、dc_read_fm11rf005。
1.3.9	2020/08/26	1,增加AOA模式打开接口; 2,兼容复合设备、阿里定制的VID/PID为257B 3031设备。
1.3.9	2020/10/30	1,优化部分示例代码; 2,增加获取IC卡卡号的接口; 3,修改获取IC卡PBOC数据异常bug。
1.4.0	2020/12/07	1,添加身份证原始数据解析接口说明; 2,优化dc_dcrf32_json_parse接口打开USB时的bug; 3,新增adb socket与PC通讯接口。
1.4.0	2020/12/10	1,添加日志服务。
1.4.1	2020/12/17	1,此说明文档改为markdown编写。
1.4.2	2021/03/17	1,添加授权功能以及更新项目导入说明; 2,添加设置SDK调用系统语言环境。
1.4.3	2021/3/24	1,添加智能终端上下电操作接口说明。
1.4.4	2021/6/4	1、添加读取非接触式三代卡卡片识别码接口。
1.4.5	2021/7/13	1、更新部分流程接口调用。
1.4.6	2021/7/21	1、增加捕获处理由于身份证图片解析时导致库Crash流程。
1.4.7	2021/08/25	1、修改远程maven仓库地址; 2、修改部分接口调用示例。
1.4.8	2022/01/07	1、修改dc_cpuapdusourceEXT_hex接口说明
1.4.9	2022/01/07	1、兼容串口921600波特率
1.5.0	2022/08/08	1、删除M1卡装载秘钥流程; 2、将验证M1卡密码的接口改为dc_authentication_pass。
1.5.1	2022/09/29	删除egapp社保卡调用示例
1.5.2	2022/10/26	扫码调用示例代码增加扫到结果后转换数据
1.5.3	2023/02/09	增加15693协议卡透传接口
1.5.4	2023/04/19	1.增加读卡器是上下电操作; 2.增加串口路径说明
1.5.5	2023/08/09	增加NFC卡读写卡接口

版本号	修改时间	修改内容
1.5.6	2023/09/28	1.增加获取身份证卡片种类接口; 2.增加新版 解析外国人永久居留证（新版）文字信息，获取相应的条目。 3.身份证源码解析借口逻辑修改
1.5.7	2023/11/21	增加继电器接口

3、工作流

· 简易工作流程图



二、项目导入

Step 1

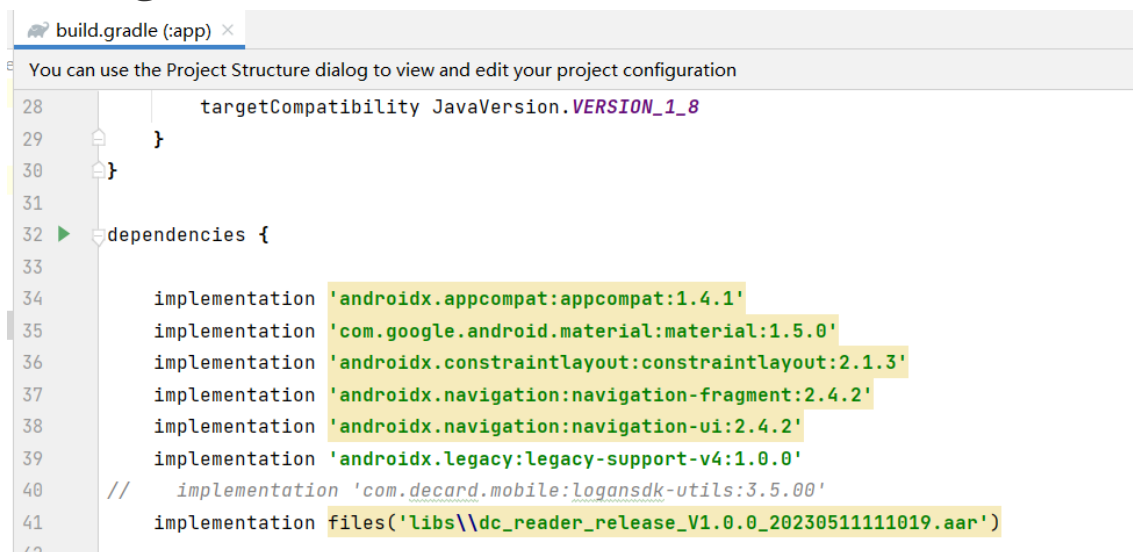
- 通常拿到的SDK压缩包名称为：
Android_sdk_releasex.xx



Step 2

这里以android studio 为默认IDE进行说明。注：步骤一、二根据具体SDK进行接入。

- 步骤1：新建项目，拷贝
dc_reader_release_V1.0.0_xxxxxxxxxxxxxxxxx.aar文件
到“app\libs”目录下，拷贝完成后，在app目录下的
build.gradle进行aar文件依赖



- 步骤2：AndroidManifest.xml中添加相关权限；

```
<uses-permission  
android:name="android.permission.INTERNET" />  
<uses-permission  
android:name="android.permission.WRITE_EXTERNAL_STORAGE" />  
<uses-permission  
android:name="android.permission.READ_EXTERNAL_STORAGE" />  
<uses-permission  
android:name="android.permission.READ_PHONE_STATE" />
```

三、智能终端

1、智能终端端口打开示例

1.1 Z90N、P18F智能终端

```
//返回值大于0成功，其他失败  
int openResult =  
BasicOper.dc_open("COM", null,  
"/dev/ttyHSL1", 115200);
```

1.2 P18Q-X1智能终端

```
//返回值大于0成功，其他失败
int openResult =
BasicOper.dc_open("COM", null,
"/dev/dc_spi32765.0", 115200);
```

四、读卡器上下电操作

1、Z90N, M200, P18F上下电操作

- 上电操作

```
DecardPowerApi decardPowerApi=new
DecardPowerApi(getBaseContext());
    if
(decardPowerApi.getCardReaderState()==
=0){

    decardPowerApi.setCardReaderState(1)
;
    }else{
        Toast.makeText(this, "上电成
功", Toast.LENGTH_SHORT).show();
    }
```

- 下电操作

```
DecardPowerApi decardPowerApi=new
DecardPowerApi(getBaseContext());
    if
(decardPowerApi.getCardReaderState()=
=1){

    decardPowerApi.setCardReaderState(0)
;
    }else{
        Toast.makeText(this, "下电成
功", Toast.LENGTH_SHORT).show();
    }
}
```

2、P10平板上下电操作

- P10平板上电操作

boolean

```
result=IoControlUtils.getInstance().setIoStatus(IoControlUtils.IC_CARDS_PATH,IoControlUtils.IOSTATUS.ENABLE);
IoControlUtils.getInstance().setIoStatus(IoControlUtils.POGO_A_PORT_PATH,IoControlUtils.IOSTATUS.ENABLE);
IoControlUtils.getInstance().setIoStatus(IoControlUtils.POGO_B_PORT_PATH,IoControlUtils.IOSTATUS.ENABLE);
IoControlUtils.getInstance().setIoStatus(IoControlUtils.SMART_READER_PATH,IoControlUtils.IOSTATUS.ENABLE);
if (result){
    Toast.makeText(this, "读卡器上电成功", Toast.LENGTH_SHORT).show();
}else{
    Toast.makeText(this, "读卡器上电失败", Toast.LENGTH_SHORT).show();
}
```

- P10平板下电操作

boolean

```
result=IoControlUtils.getInstance().setIoStatus(IoControlUtils.IC_CARDS_PATH,IoControlUtils.IOSTATUS.DISABLE);
IoControlUtils.getInstance().setIoStatus(IoControlUtils.POGO_A_PORT_PATH,IoControlUtils.IOSTATUS.DISABLE);
IoControlUtils.getInstance().setIoStatus(IoControlUtils.POGO_B_PORT_PATH,IoControlUtils.IOSTATUS.DISABLE);
IoControlUtils.getInstance().setIoStatus(IoControlUtils.SMART_READER_PATH,IoControlUtils.IOSTATUS.DISABLE);
if (result){
    Toast.makeText(this, "读卡器下电成功", Toast.LENGTH_SHORT).show();
}else{
    Toast.makeText(this, "读卡器下电失败", Toast.LENGTH_SHORT).show();
}
```

五、调用示例

1、设置SDK调用系统语言环境

```
/**
 * @param code
 * @return 0 :成功； 其他值:失败
 * @brief 设置SDK调用系统语言环境。
 * @n 0: 中文(默认)
 * @n 1: 英文
 */
BasicOper.dc_setLanguageEnv(1);
```

2、端口操作

打开

2.1 USB

使用Android USB API。

//向系统申请使用**USB**权限,此过程为异步,建议放在程序启动时调用。

```
BasicOper.dc_AUSB_ReqPermission(this);
```

//打开端口, **usb**模式, 打开之前必须确保已经获取到**USB**权限, 返回值为设备句柄号。

```
int devHandle =
BasicOper.dc_open("AUSB",this,"",0);
if(devHandle>0){
    Log.d("open","dc_open success
devHandle = "+devHandle);
}
```

使用LibUSB API。

//libusb使用之前确保设备已成功root,并且赋予应用永久使用超级管理员权限。

```
int devHandle =  
BasicOper.dc_open("LUSB",null,"",0);  
//返回值为设备句柄号。  
if(devHandle>0){  
    Log.d("open","dc_open success  
devHandle = "+devHandle);  
}
```

2.2 串口

//串口使用之前确保设备支持串口,并且已知设备串口路径。

```
int devHandle =  
BasicOper.dc_open("COM",null,"/dev/ttyS  
0",115200);  
//返回值为设备句柄号。  
if(devHandle>0){  
    Log.d("open","dc_open success  
devHandle = "+devHandle);  
}
```

2.3 蓝牙

使用经典蓝牙API。


```
//使用之前确保设备支持蓝牙。
int openResult =
BasicOper.dc_open("BT",this,"11:22:33:4
4:55:66",0);
//返回值为设备句柄号。
if(devHandle>0){
Log.d("open","dc_open success devHandle
="+devHandle);
}
```

使用低功耗蓝牙BLE API

```
//BLE 使用之前确保设备支持BLE。
int openResult =
BasicOper.dc_open("BLE",this,"11:22:33:
44:55:66",0);
//返回值为设备句柄号。
if(devHandle>0){
    Log.d("open","dc_open success
devHandle="+devHandle);
}
```

2.4串口路径

```
P18F("P18F")--串口路径: /dev/ttyHSL1
F11("f11_x1")--串口路径: /dev/ttyUSB0
Z90N("Z90N")--串口路径: /dev/ttyHSL1
P18Q("p18q_dual")--串口路
径: /dev/dc_spi32765.0
M200("M200-Q3")--串口路径: /dev/ttyHSL1
P10("P10平板设备")--串口路径: /dev/ttyS0
```

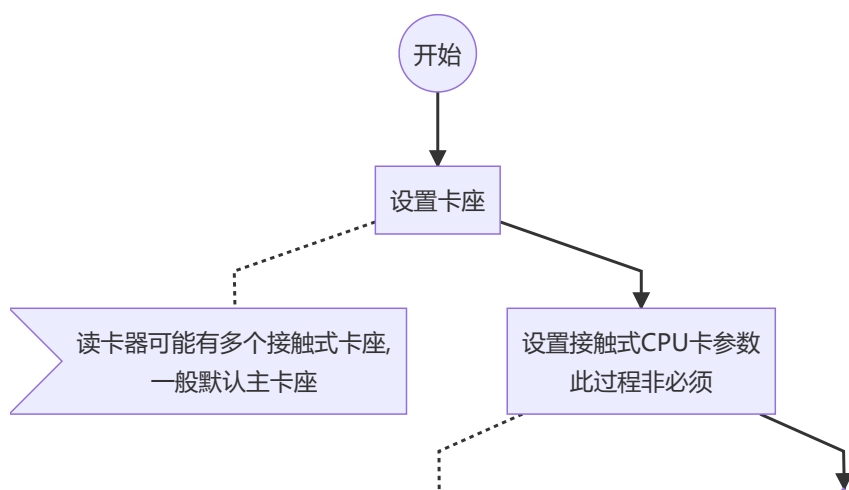
关闭

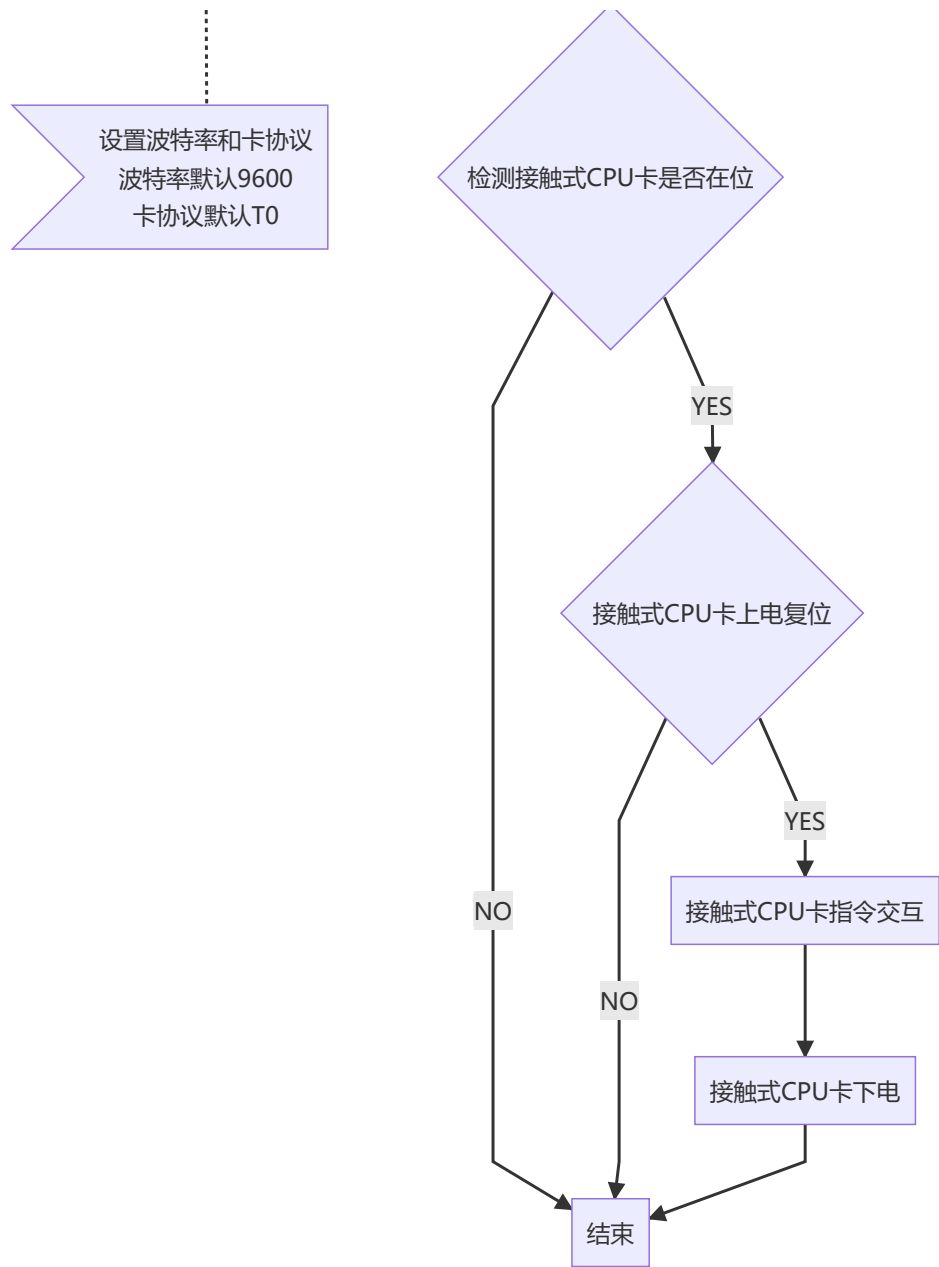
```
//关闭端口，释放资源
BasicOper.dc_exit();
```

3、卡型操作

1、接触式CPU卡

1.1 接触式CPU卡简易流程图





1.2示例代码

- 设置卡座

//设置卡座

```
String result = BasicOper.dc_setcpu(0);
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_setcpu","success");
}else{
    Log.d("dc_setcpu","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 设置接触式CPU卡参数

//设置接触式CPU卡参数

```
String result =
BasicOper.dc_setcpupara(0, 0x00,0x5C);
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_setcpupara","接触式CPU卡参
数设置成功");
}else{
    Log.d("dc_setcpupara","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 检测接触式CPU卡是否在位

```
//检测接触式卡是否在位
String result =
BasicOper.dc_card_status();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_card_status","卡片存在");
}else{
    Log.d("dc_card_status","error code
= "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 接触式CPU卡复位

```
//接触式CPU卡复位
String result =
BasicOper.dc_cpureset_hex();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_cpureset_hex","卡片复位成
功, ATR/ATS = "+resultArr[1]);
}else{
    Log.d("dc_cpureset_hex","error code
= "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 接触式CPU卡指令交互

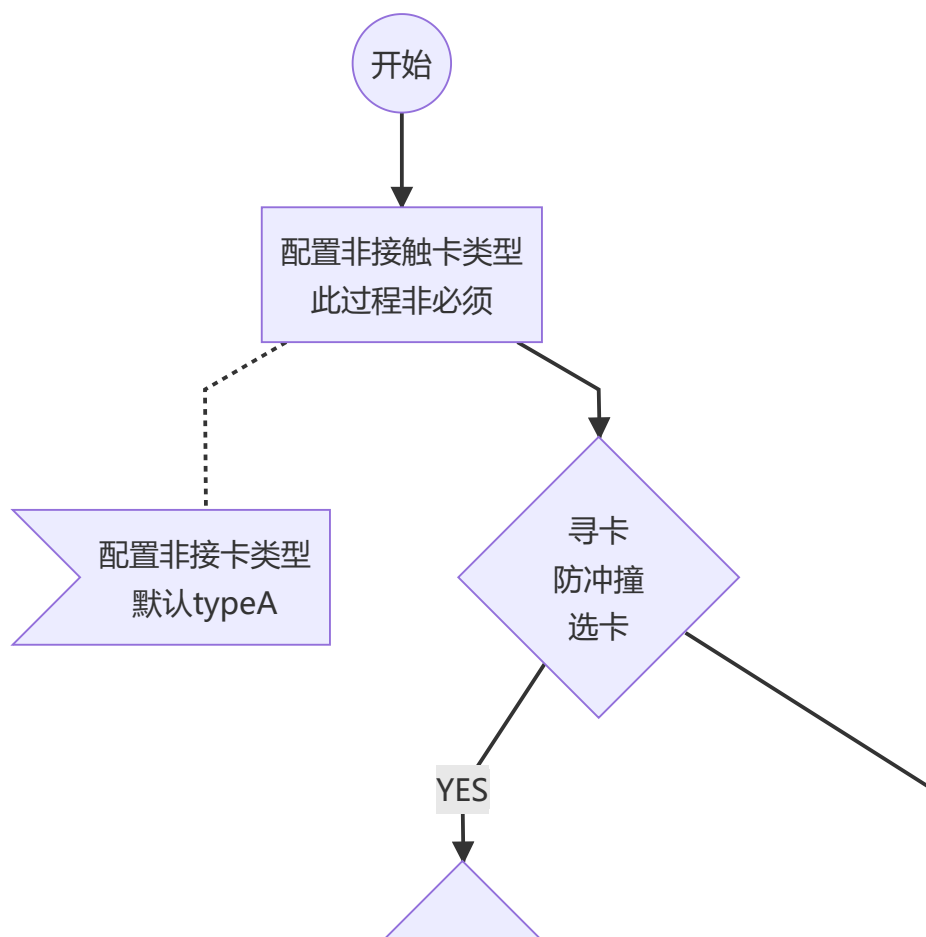
```
//接触式CPU卡指令交互
String result =
BasicOper.dc_cpuapduInt_hex("0084000008
");
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_cpuapduInt_hex","success
response = "+resultArr[1]);
}else{
    Log.d("dc_cpuapduInt_hex","error
code = "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

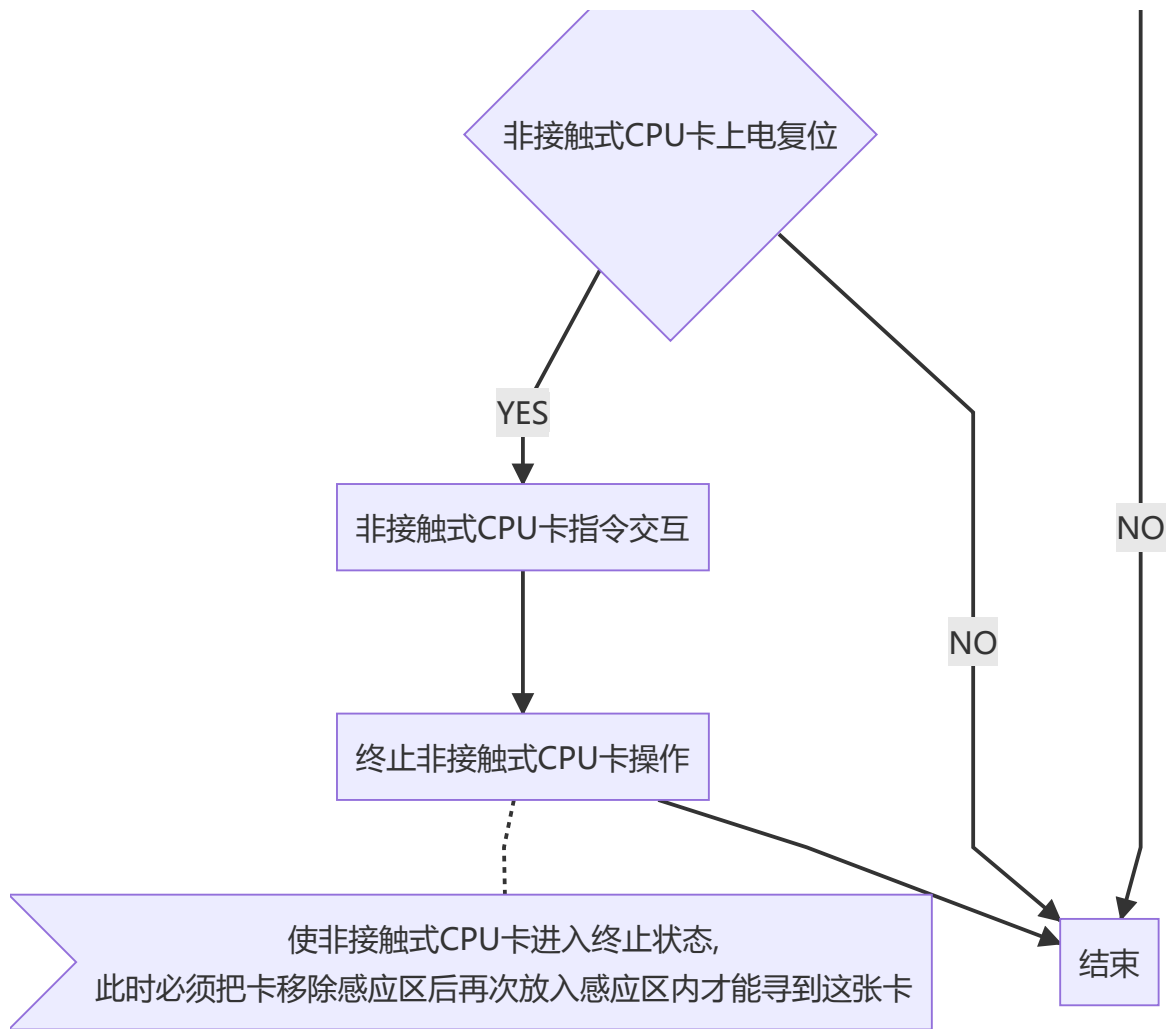
- 接触式CPU卡下电

```
//对当前卡座CPU卡进行下电操作
String result = BasicOper.dc_cpudown();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_cpudown","success");
}else{
    Log.d("dc_cpudown","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

2、非接CPU卡

2.1Type A卡简易流程图





2.2示例代码

- 射频复位

```
//射频复位
String result = BasicOper.dc_reset();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_reset","success");
}else{
    Log.d("dc_reset","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 获取接触卡类型

```
//获取感应区中存在的非接触卡类型
String result =
BasicOper.dc_card_exist();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_card_exist","success card
type = " + resultArr[1]);
    if(resultArr[1].equals("1010")){
        Log.d("dc_card_exist","表示感应区
存在激活前的Type A CPU卡或Type B CPU卡");
    }else
    if(resultArr[1].equals("1001")){
```

```
        Log.d("dc_card_exist", "表示已刷磁  
卡");  
    }else  
if(resultArr[1].equals("1002")){  
        Log.d("dc_card_exist", "表示卡座存  
在接触式CPU卡");  
    }else  
if(resultArr[1].equals("1004")){  
        Log.d("dc_card_exist", "表示卡座存  
在接触式未知卡");  
    }else  
if(resultArr[1].equals("1008")){  
        Log.d("dc_card_exist", "表示感应区  
存在身份证");  
    }else  
if(resultArr[1].equals("1020")){  
        Log.d("dc_card_exist", "表示感应区  
存在激活后的Type A CPU卡或Type B CPU卡");  
    }else  
if(resultArr[1].equals("1040")){  
        Log.d("dc_card_exist", "表示感应区  
存在激活前的M1卡");  
    }else  
if(resultArr[1].equals("1100")){  
        Log.d("dc_card_exist", "表示感应区  
存在激活前的多张Type A CPU卡");
```

```
    }else
    if(resultArr[1].equals("1200")){
        Log.d("dc_card_exist", "表示感应区
        存在激活前的多张M1卡");
    }else
    if(resultArr[1].equals("1400")){
        Log.d("dc_card_exist", "表示感应区
        同时存在激活前的Type A CPU卡和M1卡");
    }
}else{
    Log.d("dc_card_exist", "error code =
    "+resultArr[0] +" error msg =
    "+resultArr[1] );
}
```

- 配置非接触卡类型

```
//配置非接触卡类型
String result =
BasicOper.dc_config_card(0x00);
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_config_card","success ");
}else{
    Log.d("dc_config_card","error code
= "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 寻卡请求、防卡冲突、选卡操作

支持ISO 14443 Type A类型卡片的寻卡请求

```
//寻卡请求、防卡冲突、选卡操作
String result =
BasicOper.dc_card_hex(0x01);
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_card_hex","success card
sn = " + resultArr[1]);
}else{
    Log.d("dc_card_hex","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 非接触式CPU卡复位

```
//非接触式CPU卡复位
String result =
BasicOper.dc_pro_resethex();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_pro_resethex","success
ATR/ATS = " + resultArr[1]);
}else{
    Log.d("dc_pro_resethex","error code
= "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 非接触式CPU卡指令交互

```

//非接触式CPU卡指令交互
//由于底层C语言采用无符号类型unsign char ,
超过255长度的apdu使用此接口可能会出现返回成功，但是apdu为空的问题，此情况请使用接口
dc_procommandInt_hex
String result =
BasicOper.dc_procommandInt_hex
("0084000008",7);
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_procommandInt_hex
","success reponse apdu = " +
resultArr[1]);
}else{
    Log.d("dc_procommandInt_hex
","error code = "+resultArr[0] +" error
msg = "+resultArr[1] );
}

```

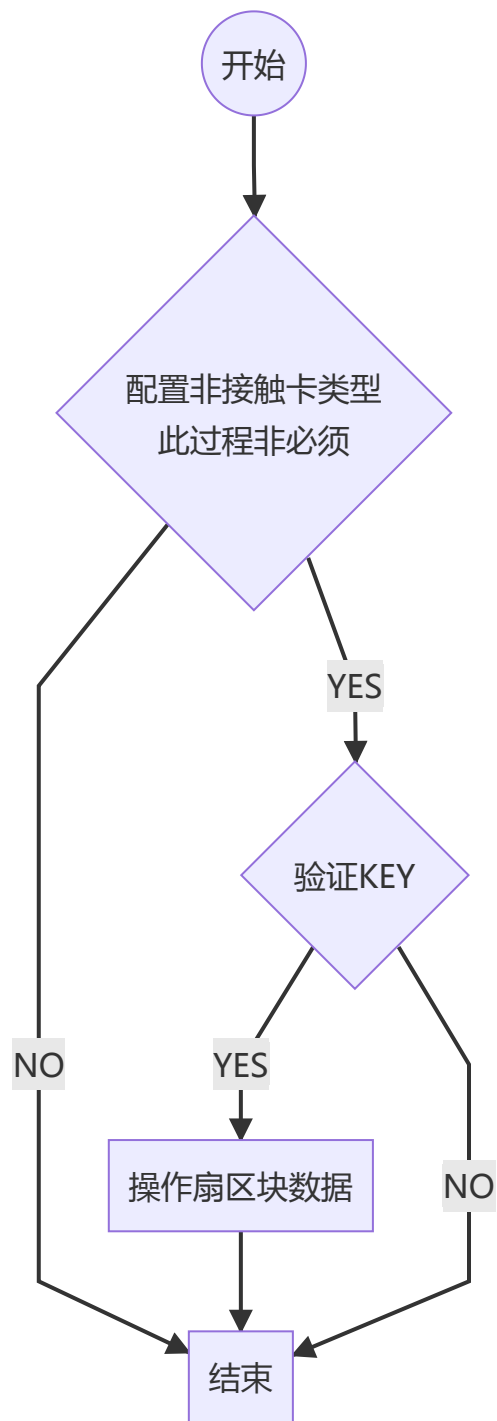
• 终止非接触式CPU卡操作

使非接触式CPU卡进入终止状态，此时必须把卡移出感应区后再次放入感应区才能寻到这张卡


```
// 终止非接触式CPU卡操作
String result =
BasicOper.dc_pro_halt();
String[] resultArr =
result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_pro_halt","success");
}else{
    Log.d("dc_pro_halt","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

3、M1卡

3.1M1卡简易流程图



3.2示例代码

- 寻卡，防冲撞，选卡 (ISO 14443 Type A)

```
// 寻卡,防冲撞,选卡三合一
String result = null;
String[] resultArr = null;
result = BasicOper.dc_card_n_hex(1);
resultArr = result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_card_hex","success 卡片序
列号: "+resultArr[1]);
}else{
    Log.d("dc_card_hex","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 验证M1卡密码

```
//验证密钥(使用设备内部装载的密码来验证M1卡密码)
String result = null;
String[] resultArr = null;
//当没有装载密钥时，使用默认密钥进行验证
result =
BasicOper.dc_authentication_pass(0,1,"FFFFFFFFFFFFFF");
resultArr = result.split("\\|",-1);
if(resultArr[0].equals("0000")){

    Log.d("dc_authentication","success");
}else{
    Log.d("dc_authentication","error
code = "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

- 读卡数据

//读卡片扇区块数据

```
String result = null;
String[] resultArr = null;
result = BasicOper.dc_read_hex(4);
resultArr = result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_read_hex","success data :
    "+resultArr[1]);
}else{
    Log.d("dc_read_hex","error code =
    "+resultArr[0] +" error msg =
    "+resultArr[1] );
}
```

- 写卡数据

```
//写数据到卡片扇区块  
String result = null;  
String[] resultArr = null;  
result =  
BasicOper.dc_write_hex(4,"00000000000000  
0000000000000000000000000000000000000001");  
resultArr = result.split("\\|",-1);  
if(resultArr[0].equals("0000")){  
    Log.d("dc_write_hex","success");  
}else{  
    Log.d("dc_write_hex","error code =  
"+resultArr[0] +" error msg =  
"+resultArr[1] );  
}
```

- ## • 终止卡操作

```
//终止卡操作
String result = null;
String[] resultArr = null;
result = BasicOper.dc_halt();
resultArr = result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_halt","success");
}else{
    Log.d("dc_halt","error code =
"+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

4、磁条卡

4.1同步方式

```
//读取磁条卡的数据，支持1、2、3轨道
String result = null;
String[] resultArr = null;
result =
BasicOper.dc_readmagcardall(10); //超时等
待刷卡10秒，获取到磁道数据立即返回。
resultArr = result.split("\\|",-1);
if(resultArr[0].equals("0000")){
    Log.d("dc_readmagcardall","success
");
    Log.d("dc_readmagcardall","success
track1 = "+resultArr[1]);
    Log.d("dc_readmagcardall","success
track2 = "+resultArr[2]);
    Log.d("dc_readmagcardall","success
track3 = "+resultArr[3]);
}else{
    Log.d("dc_readmagcardall","error
code = "+resultArr[0] +" error msg =
"+resultArr[1] );
}
```

4.2异步方式

```
//进入读磁条状态
String startMag =
BasicOper.dc_startreadmag();
```



```
while (true){
    //读取磁条卡信息
    String readMag =
BasicOper.dc_readmag();
    Log.d(TAG, "BasicOper.dc_readmag: "
+ readMag);
    String readArr[] =
readMag.split("\\|",-1);
    if (readArr[0].equals("0001") ||
readArr[0].equals("0000")) {
        if (readArr[0].equals("0000"))
{
            Log.d(TAG,
"BasicOper.dc_readmag  result: $ret");
        }
        //退出读取磁条卡状态
        readMag =
BasicOper.dc_stopreadmag();
        Log.d(TAG,
"BasicOper.dc_stopreadmag  result:
$ret");
        readMag =
BasicOper.dc_startreadmag();
        Log.d(TAG,
"BasicOper.dc_startreadmag  result:
$ret");
    }
}
```

```
        SystemClock.sleep(200);  
    }
```

5、身份证

5.1 读取身份证信息

```
IDCard idCard =  
BasicOper.dc_SamAReadCardInfo(1);  
if(idCard!=null) {  
    Log.d(LOG, "dc_SamAReadCardInfo ret  
= " + idCard.toString());  
    ImageView imageView = (ImageView)  
findViewById(R.id.ImageView);  
  
    imageView.setImageBitmap(BitmapFactory  
.decodeByteArray(idCard.getPhotoData(),  
0, idCard.getPhotoData().length));  
}
```

5.2 读取身份证原始数据信息并进行解析

```
private static int photoLen = 1024;  
private static byte[] photo_len =  
DataDealUtil.intToByteArray(photoLen);  
private static byte[] photo = new  
byte[1024];  
private static int textLen = 256;
```

```
private static byte[] text_len =
DataDealUtil.intToByteArray(textLen);
private static byte[] text = new
byte[256];
private static int fingerprintLen =
1024;
private static byte[] fingerprint_len =
DataDealUtil.intToByteArray(fingerprint
Len);
private static byte[] fingerprint = new
byte[1024];
private static int extraLen = 70;
private static byte[] extra_len =
DataDealUtil.intToByteArray(extraLen);
private static byte[] extra = new
byte[70];
//读取身份证的原始信息数据
public static short
getSamAReadCardInfoRaw(int tyoe) {
    short res =
BasicOper.dc_SamAReadCardInfo(type,
text_len, text, photo_len, photo,
fingerprint_len, fingerprint,
extra_len, extra);
    return res;
}
//解析文字信息
```

```
private static String
dc_IdCardReadCardInfoString(int
textLen, byte[] text) {
    String result = "";
    textLen =
DataDealUtil.byteArrayToInt(text_len);
    //中华人民共和国居民身份证文字信息
    byte[] name = new byte[30];
    byte[] sex = new byte[8];
    byte[] nation = new byte[64];
    byte[] birthday = new byte[16];
    byte[] address = new byte[144];
    byte[] id = new byte[36];
    byte[] department = new
byte[30];
    byte[] startTime = new
byte[16];
    byte[] endTime = new byte[16];
    byte[] reserved = new byte[70];

    //外国人永久居留证文字信息(2017版本)
    byte[] Foreign_name = new
byte[244];
    byte[] Foreign_sex = new
byte[8];
```

```
        byte[] Foreign_number = new
byte[64];
        byte[] Foreign_citizenship =
new byte[16];
        byte[] Foreign_chinese_name =
new byte[64];
        byte[] Foreign_expire_start_day
= new byte[36];
        byte[] Foreign_expire_end_day =
new byte[36];
        byte[] Foreign_birth_day = new
byte[36];
        byte[] Foreign_version_number =
new byte[12];
        byte[] Foreign_department_code
= new byte[20];
        byte[] Foreign_type_sign = new
byte[8];
        byte[] Foreign_reserved = new
byte[16];
```

//港澳台居民居住证文字信息

```
        byte[]
Hongkong_macao_taiwan_name = new
byte[64];
```

```
        byte[]
Hongkong_macao_taiwan_sex = new
byte[8];
        byte[]
Hongkong_macao_taiwan_reserved = new
byte[12];
        byte[]
Hongkong_macao_taiwan_birth_day = new
byte[36];
        byte[]
Hongkong_macao_taiwan_address = new
byte[144];
        byte[]
Hongkong_macao_taiwan_number = new
byte[76];
        byte[]
Hongkong_macao_taiwan_department = new
byte[64];
        byte[]
Hongkong_macao_taiwan_expire_start_day
= new byte[36];
        byte[]
Hongkong_macao_taiwan_expire_end_day =
new byte[36];
        byte[]
Hongkong_macao_taiwan_pass_number = new
byte[40];
```

```
        byte[]
Hongkong_macao_taiwan_sign_count = new
byte[12];
        byte[]
Hongkong_macao_taiwan_reserved2 = new
byte[16];
        byte[]
Hongkong_macao_taiwan_type_sign = new
byte[8];
        byte[]
Hongkong_macao_taiwan_reserved3 = new
byte[16];

//外国人永久居留证文字信息(新版本)
        byte[] chinese_name = new
byte[64];
        byte[] Foreign_new_sex = new
byte[8];
        byte[] renew_count = new
byte[12];
        byte[] birth_day = new
byte[36];
        byte[] english_name = new
byte[255 * 2];
        byte[] id_number = new
byte[76];
```

```
        byte[] new_reserved = new
byte[64];
        byte[] expire_start_day = new
byte[36];
        byte[] expire_end_day = new
byte[36];
        byte[] english_name_ex = new
byte[48];
        byte[] citizenship = new
byte[16];
        byte[] type_sign = new byte[8];
        byte[] prev_related_info = new
byte[16];
        byte[] old_id_number = new
byte[64];
```

```
        short s =
BasicOper.dc_GetIdCardType(textLen,
text);
```

```
        Log.d("TAG", "结果" + s);
```

```
        if (s >= 0) {
```

```
            if (s == 1) {
```

```
                //表示外国人永久居留证
```

(2017版)


```
        short res =
BasicOper.dc_ParseTextInfoForForeigner(
2, textLen, text, Foreign_name,
Foreign_sex, Foreign_number,
Foreign_citizenship,
Foreign_chinese_name,
Foreign_expire_start_day,
Foreign_expire_end_day,
Foreign_birth_day,
Foreign_version_number,
Foreign_department_code,
Foreign_type_sign, Foreign_reserved);
        Log.d("TAG", "Read info
dc_ParseTextInfoForForeigner: " + res);
        if (res == 0) {
            String foreign_name
= new String(Foreign_name).trim() +
"\\|";
            String foreign_sex
= dc_ParseOtherInfo(0x20,
Foreign_sex).split("\\|")[1] + "|";
            String
foreign_number = new
String(Foreign_number).trim() + "|";
```

String

```
foreign_citizenship =  
dc_ParseOtherInfo(0x22,  
Foreign_citizenship).split("\\|")[1] +  
"|";
```

String

```
foreign_chinese_name = new  
String(Foreign_chinese_name).trim() +  
"|";
```

String

```
foreign_expire_start_day = new  
String(Foreign_expire_start_day).trim()  
+ "|";
```

String

```
foreign_expire_end_day = new  
String(Foreign_expire_end_day).trim() +  
"|";
```

String

```
foreign_birth_day = new  
String(Foreign_birth_day).trim() + "|";
```

String

```
foreign_version_number = new  
String(Foreign_version_number).trim() +  
"|";
```

```

        String
foreign_department_code = new
String(Foreign_department_code).trim()
+ "|";

        String
foreign_type_sign = new
String(Foreign_type_sign).trim() + "|";

        String
foreign_reserved = new
String(Foreign_reserved).trim();

        result = "0000|1|"
+ foreign_name + foreign_sex +
foreign_number + foreign_citizenship +
foreign_chinese_name +
foreign_expire_start_day +
foreign_expire_end_day +
foreign_birth_day +
foreign_version_number +
foreign_department_code +
foreign_type_sign + foreign_reserved;

        Log.d("TAG", "Read
info dc_ParseTextInfoForForeigner
Result: " + result);

        return result;
    } else {

```

```

        return "0001|解析失败";
    }
} else if (s == 2) {
    //表示港澳台居民居住证
    short res =
BasicOper.dc_ParseTextInfoForHkMoTw(2,
textLen, text,
Hongkong_macao_taiwan_name,
Hongkong_macao_taiwan_sex,
Hongkong_macao_taiwan_reserved,
Hongkong_macao_taiwan_birth_day,
Hongkong_macao_taiwan_address,
Hongkong_macao_taiwan_number,
Hongkong_macao_taiwan_department,
Hongkong_macao_taiwan_expire_start_day,
Hongkong_macao_taiwan_expire_end_day,
Hongkong_macao_taiwan_pass_number,
Hongkong_macao_taiwan_sign_count,
Hongkong_macao_taiwan_reserved2,
Hongkong_macao_taiwan_type_sign,
Hongkong_macao_taiwan_reserved3);
    Log.d("TAG", "Read info
dc_ParseTextInfoForHkMoTw: " + res);
    if (res == 0) {

```

```
String HMT_name =  
new  
String(Hongkong_macao_taiwan_name).trim  
() + "|";  
  
String HMT_sex =  
dc_ParseOtherInfo(0x20,  
Hongkong_macao_taiwan_sex).split("\\|")  
[1].trim() + "|";  
  
String HMT_reserved  
= new  
String(Hongkong_macao_taiwan_reserved).  
trim() + "|";  
  
String  
HMT_birth_day = new  
String(Hongkong_macao_taiwan_birth_day)  
.trim() + "|";  
  
String HMT_address  
= new  
String(Hongkong_macao_taiwan_address).t  
rim() + "|";  
  
String HMT_number =  
new  
String(Hongkong_macao_taiwan_number).tr  
im() + "|";
```

String

```
HMT_department = new  
String(Hongkong_macao_taiwan_department  
) .trim() + "|";
```

String

```
HMT_expire_start_day = new  
String(Hongkong_macao_taiwan_expire_sta  
rt_day) .trim() + "|";
```

String

```
HMT_expire_end_day = new  
String(Hongkong_macao_taiwan_expire_end  
_day) .trim() + "|";
```

String

```
HMT_pass_number = new  
String(Hongkong_macao_taiwan_pass_numbe  
r) .trim() + "|";
```

String

```
HMT_sign_count = new  
String(Hongkong_macao_taiwan_sign_count  
) .trim() + "|";
```

String

```
HMT_reserved2 = new  
String(Hongkong_macao_taiwan_reserved2)  
 .trim() + "|";
```

String

```
HMT_type_sign = new  
String(Hongkong_macao_taiwan_type_sign)  
.trim() + "|";
```

String

```
HMT_reserved3 = new  
String(Hongkong_macao_taiwan_reserved3)  
.trim();
```

```
result = "0000|2|"  
+ HMT_name + HMT_sex + HMT_reserved +  
HMT_birth_day + HMT_address +  
HMT_number + HMT_department +  
HMT_expire_start_day +  
HMT_expire_end_day + HMT_pass_number +  
HMT_sign_count + HMT_reserved2 +  
HMT_type_sign + HMT_reserved3;
```

```
Log.d("TAG", "Read  
info dc_ParseTextInfoForHkMoTw Result:  
" + result);
```

```
return result;  
} else {  
return "0001|解析失  
败";
```

```
}  
} else if (s == 3) {  
//表示外国人永久居留证（新  
版）
```

//外国人永久居留证

short res =

```
BasicOper.dc_ParseTextInfoForNewForeigner(2, textLen, text, chinese_name, Foreign_new_sex, renew_count, birth_day, english_name, id_number, new_reserved, expire_start_day, expire_end_day, english_name_ex, citizenship, type_sign, prev_related_info, old_id_number);
```

```
Log.d("TAG", "Read info dc_ParseTextInfoForForeigner: " + res);
```

```
if (res == 0) {
```

```
String
```

```
foreign_Engoinsh = new
```

```
String(chinese_name).trim() + "\\|";
```

```
String foreign_sex
```

```
= dc_ParseOtherInfo(0x20,
```

```
Foreign_new_sex).split("\\|")[1] + "|";
```

```
String
```

```
foreign_number = new
```

```
String(renew_count).trim() + "|";
```

```
String
```

```
foreign_citizenship = new
```

```
String(birth_day).trim() + "|";
```



```
        String
foreign_chinese_name = new
String(english_name).trim() + "|";
        String
foreign_expire_start_day = new
String(id_number).trim() + "|";
        String
foreign_expire_end_day = new
String(new_reserved).trim() + "|";
        String
foreign_birth_day = new
String(expire_start_day).trim() + "|";
        String
foreign_version_number = new
String(expire_end_day).trim() + "|";
        String
foreign_department_code = new
String(english_name_ex).trim() + "|";
        String Citizenship
= new String(citizenship).trim() + "|";
        String TypeSign =
new String(type_sign).trim() + "|";
        String
prevRelatedInfo = new
String(prev_related_info).trim() + "|";
        String oldINumber =
new String(old_id_number).trim();
```

```

        result = "0000|1|"
+ foreign_Engoinsh + foreign_sex +
foreign_number + foreign_citizenship +
foreign_chinese_name +
foreign_expire_start_day +
foreign_expire_end_day +
foreign_birth_day +
foreign_version_number +
foreign_department_code + Citizenship +
TypeSign + prevRelatedInfo +
oldINumber;

        Log.d("TAG", "Read
info dc_ParseTextInfoForForeigner
Result: " + result);

        return result;
    } else {
        return "0001|解析失
败";
    }
} else {
    //中华人民共和国居民身份证
    short res =
BasicOper.dc_ParseTextInfo(2, textLen,
text, name, sex, nation, birthday,
address, id, department, startTime,
endTime, reserved);

```

```
        Log.d("TAG", "Read info  
dc_ParseTextInfo: " + res);  
        if (res == 0) {  
            String Name = new  
String(name).trim() + "|";  
            String Sex =  
dc_ParseOtherInfo(0x20,  
sex).split("\\|")[1] + "|";  
            Log.d("TAG",  
"#####" + Sex);  
            String Nation =  
dc_ParseOtherInfo(0x21,  
nation).split("\\|")[1] + "|";  
            Log.d("TAG",  
"#####" + Nation);  
            String Birthday =  
new String(birthday).trim() + "|";  
            String Address =  
new String(address).trim() + "|";  
            String Id = new  
String(id).trim() + "|";  
            String Department =  
new String(department).trim() + "|";  
            String Start_time =  
new String(startTime).trim() + "|";  
            String End_time =  
new String(endTime).trim() + "|";
```

```

        String Reserved =
new String(reserved).trim();
        result = "0000|0|"
+ Name + Sex + Nation + Birthday +
Address + Id + Department + Start_time
+ End_time + Reserved;
        Log.d("TAG", "Read
info dc_ParseTextInfo Result: " +
result);

        return result;
    } else {
        return "0001|解析失
败";
    }
}
} else {
    Log.d("TAG", "读取失败");
    return "读取失败";
}

}

// 解析其他信息 0x20:性别 0x21: 民族 0x22:
国籍
// @param info 对应数据
private static String
dc_ParseOtherInfo(int type, byte[]
info) {

```

```

        byte[] out_info = new byte[16];
        String result = "";
        short res =
BasicOper.dc_ParseOtherInfo(type, info,
out_info);
        if (res == 0) {
            result = new
String(out_info).trim();
            Log.d(TAG, "dc_ParseOtherInfo:
" + res + "result: " + result);
            if (result != "") {
                return "0000|" + result;
            } else {
                return "0001|解析值为空";
            }
        } else {
            return "0001|解析失败";
        }
    }
}
//解析图片数据
public static byte[]
getParsePhotoInfo(byte[] photo_len,
byte[] photo) {
    int photoLen =
DataDealUtil.byteArrayToInt(photo_len);
    int PhotoLen1 = 65536;

```

```

        byte[] Photo_len1 =
DataDealUtil.intToByteArray(PhotoLen1);
        byte[] photoData = new byte[65536];
        short ret =
BasicOper.dc_ParsePhotoInfo(1,
photoLen, photo, Photo_len1,
photoData);
        if (ret == 0L) {
            return photoData;
        }
        return "图片信息解析失
败!".getBytes();
    }
    //解析照片信息并保存到指定目录 此接口保存文件
    时需要读写权限
    public static int
    dc_ParsePhotoInfo(String path) {
        //"/mnt/sdcard/dc_photo.bmp"
        if
        (DataDealUtil.byteArrayToInt(photo) ==
0) {
            return -2;
            //Please read the ID
            first
        }
        photoLen =
DataDealUtil.byteArrayToInt(photo_len);

```

```

byte[] p1 = new byte[1];
//此接口保存文件时需要读写权限
short res =
BasicOper.dc_ParsePhotoInfo(0,
photoLen, photo, p1, path.getBytes());
Log.d("BasicOperYH", "storage
dc_ParsePhotoInfo: " + res);
if (res == 0) {
    return 0;
    //success
}
return -1;
//failure
}

```

6、二维码

6.1二维码扫描

```

String start_scan_2D =
BasicOper.dc_Scan2DBarcodeStart(0x00);

//开启二维码扫描
Log.d(TAG,
"BasicOper.dc_Scan2DBarcodeStart;" +
start_scan_2D);

```

```
Long timeout =
System.currentTimeMillis();

//获取系统当前时间
while (start_scan_2D.split("\\|")
[0].equals("0000")) {
    String scan_info_hex =
BasicOper.dc_Scan2DBarcodeGetData();
    byte[] bytes =
HexDump.hexStringToByteArray(scan_info_
hex.split("\\|")[1]);
    String scanInfo = new
String(bytes).trim();
    //获取二维码扫描结果
    Log.d(TAG,
"BasicOper.dc_Scan2DBarcodeGetData: " +
scanInfo);
    if (scan_info_hex.split("\\|")
[0].equals("0000")) {
        scan_2D_info=new
String(HexDump.hexStringToByteArray(sca
n_2D_info.split("\\|")
[1]), "Gbk").trim();
        Log.d(TAG,
"BasicOper.dc_Scan2DBarcodeGetData: " +
scan_2D_info);
        break;
    }
}
```



```
    }  
    if (System.currentTimeMillis() -  
timeOut > 10000) {  
        //扫描超时退出扫描  
  
        Toast.makeText(getApplicationContext()  
        , "扫描超时!",  
        Toast.LENGTH_SHORT).show();  
        break;  
    }  
}  
String scan_2D_exit =  
BasicOper.dc_Scan2DBarcodeExit();  
  
//停止二维码扫描  
Log.d(TAG,  
"BasicOper.dc_Scan2DBarcodeExit: " +  
scan_2D_exit);}
```

7、4442卡

7.1检测4442卡

```
//检测4442卡
String ret = BasicOper.dc_Check_4442();
String retArr[] = ret.split("\\|",-1);
if(retArr[0].equals("0000")){
    Log.d(TAG, "check4442Card:
success: "+ret);
}else {
    Log.d(TAG, "check4442Card: error
code: "+retArr[0]+" error msg:
"+retArr[1]);
}
```

7.2验证4442卡密码

```
//验证4442卡密码
String ret =
BasicOper.dc_verifypin_4442_hex("4442卡
密码");
String retArr[] = ret.split("\\|",-1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "verifyPIN_4442:
success: "+retArr[1]);
else
    Log.d(TAG, "verifyPIN_4442: error
code: "+ret);
```

7.3 读取4442卡密码

```
//读取4442卡密码
String ret =
BasicOper.dc_readpin_4442_hex();
String retArr[] = ret.split("\\|",-1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readPin_4442_hex:
success: "+retArr[1]);
else
    Log.d(TAG, "readPin_4442_hex: error
code: "+ret);
```

7.4 读取4442卡密码计数

```
//读取4442卡的密码计数，此计数值表示可以尝试验
证密码的次数。
String ret =
BasicOper.dc_readpincount_4442();
String retArr[] = ret.split("\\|",-1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readpincount_4442:
success: "+retArr[1]);
else
    Log.d(TAG, "readpincount_4442:
error code: "+ret);
```

7.5修改4442卡密码

```
//修改4442卡密码
String ret =
BasicOper.dc_changepin_4442_hex("新密
码");
String retArr[] = ret.split("\\|",-1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "changepin_4442:
success: "+retArr[1]);
else
    Log.d(TAG, "changepin_4442: error
code: "+ret);
```

7.6读4442卡保护位

```
//读4442卡保护位
String ret =
BasicOper.dc_readprotect_4442_hex(0,
10);
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readprotect_4442:
success: " + retArr[1]);
else
    Log.d(TAG, "readprotect_4442: error
code: " + ret);
```

7.7 写4442卡保护位

```
//写4442卡保护位
String ret =
BasicOper.dc_writeprotect_4442_hex(0,
10, "写入的数据(16进制)"));
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "writeprotect_4442:
success: " + retArr[1]);
else
    Log.d(TAG, "writeprotect_4442:
error code: " + ret);
```

7.8 读4442卡数据

```
//读4442卡数据
String ret =
BasicOper.dc_read_4442_hex(0,10);
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "read_4442: success: " +
retArr[1]);
else
    Log.d(TAG, "read_4442: error code:
" + ret);
```

7.9 写4442卡数据

```
//写4442卡数据
String ret =
BasicOper.dc_write_4442_hex(0, 10, "写入
的数据 (HEX)");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "write_4442: success: "
+ retArr[1]);
else
    Log.d(TAG, "write_4442: error code:
" + ret);
```

7.10 4442卡下电操作

```
//4442卡下电操作
String ret = BasicOper.dc_down_4442();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "down_4442: success: " +
retArr[1]);
else
    Log.d(TAG, "down_4442: error code:
" + ret);
```

8、4428卡

8.1 检测4428卡

```
//检测4428卡
String ret = BasicOper.dc_Check_4428();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "check4428Card: success"
        + ret);
else
    Log.d(TAG, "check4428Card: error"
        + ret);
```

8.2 验证4428卡密码

```
//验证4428卡密码
String ret =
BasicOper.dc_verifypin_4428_hex("4428卡"
    + "密码");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "verifypin_4428: success"
        + ret);
else
    Log.d(TAG, "verifypin_4428: error"
        + ret);
```

8.3 读取4428卡密码

```
//读取4428卡密码
String ret =
BasicOper.dc_readpin_4428_hex();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readpin_4428: success "
+ ret);
else
    Log.d(TAG, "readpin_4428: error
code " + ret);
```

8.4 读取4428卡密码计数

```
//读取4428卡的密码计数，此计数值表示可以尝试验
证密码的次数。
String ret =
BasicOper.dc_readpincount_4428();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readpincount_4428:
success " + ret);
else
    Log.d(TAG, "readpincount_4428:
error code " + ret);
```


8.5 修改4428卡密码

```
//修改4428卡密码
String ret =
BasicOper.dc_changepin_4428_hex("新密
码");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "changepin_4428: success
" + ret);
else
    Log.d(TAG, "changepin_4428: error
code " + ret);
```

8.6 读4428卡保护位

```
//读4428卡保护位
String ret =
BasicOper.dc_readprotect_4428_hex(0,10)
;
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readprotect_4428:
success " + ret);
else
    Log.d(TAG, "readprotect_4428: error
code " + ret);
```

8.7 写4428卡保护位

```
//写4428卡保护位
String ret =
BasicOper.dc_writeprotect_4428_hex(0,
10, "写入的数据(16进制)"));
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "writeprotect_4428:
success " + ret);
else
    Log.d(TAG, "writeprotect_4428:
error code " + ret);
```

8.8 读4428卡数据

```
//读4428卡数据
String ret =
BasicOper.dc_read_4428_hex(0,10);
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "read_4428: success " +
ret);
else
    Log.d(TAG, "read_4428: error code "
+ ret);
```

8.9 写4428卡数据

```
//写4428卡数据
String ret =
BasicOper.dc_write_4428_hex(0, 10, "写入
的数据 (HEX)");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "write_4428: success " +
ret);
else
    Log.d(TAG, "write_4428: error code
" + ret);
```

8.10 4428卡下电操作

```
//4428卡下电操作
String ret = BasicOper.dc_down_4428();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "down_4428: success " +
ret);
else
    Log.d(TAG, "down_4428: error code "
+ ret);
```

9、Fm11rf005卡

9.1 读取卡内数据

```
//fm11rf005寻卡操作  寻卡成功返回卡片序列号
String ret =
BasicOper.dc_card_fm11rf005(0);
Log.d(TAG,
"BasicOper.dc_card_fm11rf005: " + ret);
if (ret.split("\\|", -1)
[0].equals("0000")) {
    //读取卡信息  成功返回4字节数据
    ret =
BasicOper.dc_read_fm11rf005(1);
    Log.d(TAG,
"BasicOper.dc_read_fm11rf005: " + ret);
}
```

10、102卡

10.1 检测是否存在102卡

```
//检测是否存在102卡
String ret = BasicOper.dc_check_102();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "check_102: success: " +
retArr[1]);
else
    Log.d(TAG, "check_102: error code:
" + ret);
```

10.2 校验102卡密码

```
//校验102卡密码
String ret =
BasicOper.dc_checkpass_102(0, "102卡密
码");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "checkpass_102: success:
" + retArr[1]);
else
    Log.d(TAG, "checkpass_102: error
code: " + ret);
```

10.3 修改12卡密码

```
//修改102卡密码
String ret =
BasicOper.dc_changepass_102(0, "新密
码");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "changepass_102:
success: " + retArr[1]);
else
    Log.d(TAG, "changepass_102: error
code: " + ret);
```

10.4 读102卡密码计数

```
//读102卡密码计数。
String ret =
BasicOper.dc_readcount_102(0);
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "readcount_102: success:
" + retArr[1]);
else
    Log.d(TAG, "readcount_102: error
code: " + ret);
```

10.5 读102卡数据

```
//读102卡数据
String ret = BasicOper.dc_read_102(0,
10);
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "read_102: success: " +
retArr[1]);
else
    Log.d(TAG, "read_102: error code: "
+ ret);
```

10.6 写102卡数据

```
String ret = BasicOper.dc_write_102(0,
10, "写入的数据(16进制)");
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "write_102: success: " +
retArr[1]);
else
    Log.d(TAG, "write_102: error code:
" + ret);
```

10.7 102卡下电操作

```
//102卡下电操作
String ret = BasicOper.dc_down_102();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "down_102: success: " +
retArr[1]);
else
    Log.d(TAG, "down_102: error code: "
+ ret);
```

10.8 对102卡进行熔丝操作

```
//对102卡进行熔丝操作
String ret = BasicOper.dc_fuse_102();
String retArr[] = ret.split("\\|", -1);
if (retArr[0].equals("0000"))
    Log.d(TAG, "fuse_102: success: " +
retArr[1]);
else
    Log.d(TAG, "fuse_102: error code: "
+ ret);
```

4、打印模块

1、打印图片


```
//打印图片
PrinterLogic printerLogic = new
PrinterLogic();
//打印机参数设置
String setPrint =
BasicOper.dc_setprint(0x02, 0x01, 0, 0,
10, 0x00);
Log.d("print", "BasicOper.dc_setprint:"
+ setPrint);
//打印机进纸
String enter =
BasicOper.dc_printenter(50);
Log.d("print",
"BasicOper.dc_printenter:" + enter);
//打印图片
int temp =
printerLogic.PrintLineByLinePicture(320
, 233, getResources(),
R.drawable.photo);
Log.d("print",
"PrintLineByLinePicture:" + temp);
```

2、打印字符

```
//打印机参数设置
```

```
String setPrint =
BasicOper.dc_setprint(0x02, 0x01, 0, 0,
10, 0x00);
Log.d("print", "BasicOper.dc_setprint:"
+ setPrint);
//打印机进纸设置
String enter =
BasicOper.dc_printenter(50);
Log.d("print",
"BasicOper.dc_printenter:" + enter);
String printString = "德卡科技! ";
try {
    byte[] strByte =
printString.getBytes("GBK");
    //打印字符
    String print_char =
BasicOper.dc_printcharacter(strByte);
    Log.d("print",
"BasicOper.dc_printcharacter:" +
print_char);
} catch (UnsupportedEncodingException
e) {
    e.printStackTrace();
}
```

3、打印一维码

```
//打印机参数设置
String setPrint =
BasicOper.dc_setprint(0x02, 0x02, 0, 0,
10, 0x00);
Log.d("print", "BasicOper.dc_setprint:"
+ setPrint);
//打印机进纸设置
String enter =
BasicOper.dc_printenter(50);
Log.d("print",
"BasicOper.dc_printenter:" + enter);
//一维码内容
String strTemp = "15901234618";
try {
    byte[] byteTemp =
strTemp.getBytes("GBK");
    //打印一维码
    String temp =
BasicOper.dc_printOnedimensional(50,
0x01, 0x01, byteTemp);
    Log.d("print",
"PrintLineByLinePicture:" + temp);
} catch (UnsupportedEncodingException
e) {
    e.printStackTrace();
}
```

}

六、常见问题

1、项目配置

Q1:无法正常调用jar包中的类方法?

- 请检查调用模块gradle文件中，是否正常引用了jar包。

Q2:java.lang.UnsatisfiedLinkError Cannot load library ?

- 请检查so库是否导入正确，确保在src\main\jniLibs中有对应手机cpu架构的so库，检查引用的模块gradle文件中，是否有修改 jniLibs.srcDirs 标签属性。

```
sourceSets {  
    main {  
        jniLibs.srcDirs = ['libs'] //  
加上这行的话，需要把动态库添加到此模块根目录/libs下，否则apk无法正常导入so库，出现运行时找不到so库的错误。  
    }  
}
```

Q3:清单文件中已经注册了蓝牙权限，但还是无法使用蓝牙？

- 系统6.0及以上版本，使用蓝牙必须向系统动态申请定位权限，系统会提示框，用户必须点击允许，代码参考如下：。

```
/*6.0系统需要动态检查定位权限*/
protected void checkSelfPermission(){

    if(ContextCompat.checkSelfPermission(
        this,
        Manifest.permission.ACCESS_COARSE_LOCATION) !=
        PackageManager.PERMISSION_GRANTED){

        ActivityCompat.requestPermissions(this, new String[]
        {Manifest.permission.ACCESS_COARSE_LOCATION}, 999);

        if(ActivityCompat.shouldShowRequestPermissionRationale(this,
        Manifest.permission.ACCESS_COARSE_LOCATION)){

            showSnackBar(getWindow().getDecorView(), "此软件需要获取定位权限",
            Snackbar.LENGTH_SHORT);

        }
    }
}
```

```

    }
}

/* 检查用户是否点击允许，此方法在Activity
中Override*/
@Override
public void
onRequestPermissionsResult(int
requestCode, @NonNull String[]
permissions, @NonNull int[]
grantResults) {

    super.onRequestPermissionsResult(req
uestCode, permissions, grantResults);
    if(requestCode==999){
        if(resultCode==RESULT_OK){//
用户点击允许
        }else{//用户点击拒绝
        }
    }
}
}

```

Q4:我不知道在清单文件注册哪些?

- 请参考如下(如APP需其他权限，请自行添加):

```
/*蓝牙(经典|BLE)
```

```
<uses-permission
android:name="android.permission.BLUE
TOOTH_ADMIN"></uses-permission>
/*蓝牙(经典|BLE)
<uses-permission
android:name="android.permission.BLUE
TOOTH"></uses-permission>
/*蓝牙(经典|BLE|android6.0以上)
<uses-permission
android:name="android.permission.ACCE
SS_COARSE_LOCATION"></uses-
permission>
/*蓝牙(经典|BLE|android6.0以上)
<uses-permission
android:name="android.permission.ACCE
SS_FINE_LOCATION"></uses-permission>

/*蓝牙(经典|BLE)
<uses-feature
android:name="android.hardware.BLUETOOTH_
ADMIN"/>
/*蓝牙(BLE)
<uses-feature
android:name="android.hardware.blueto
oth_le" android:required="true"/>
/*USB
```



```
<uses-feature
android:name="android.hardware.usb.ho
st" />
/*外部存储文件读写（如果需要）
<uses-permission
android:name="android.permission.WRIT
E_EXTERNAL_STORAGE"></uses-
permission>
<uses-permission
android:name="android.permission.MOUN
T_UNMOUNT_FILESYSTEMS"></uses-
permission>
```

Q5:libwlt2bmp.so: has text relocations?

- libwlt2bmp.so是公安部提供的解析身份证头像(加密过)的动态库，此库编译时的targetSdkVersion版本为21，引用此库的项目targetSdkVersion版本不能大于此动态库编译时的版本，建议设置21。

2、接口调用

Q1: APDU长度超过255，部分接口返回成功，Apdu部分却为空？

- 由于底层C语言采用无符号类型unsign char，超过255长度的apdu使用此接口可能会出现返回成功，但是apdu为空的问题，此情况：

- 非接触式CPU卡请使用接口 **dc_procommandInt_hex**
- 接触式CPU卡请使用接口 **dc_cpuapduInt_hex**

Q2: 我的Android设备串口没有操作权限，能否在打开之前修改文件权限？

- java开发参考如下（代码参考 需翻墙 from: <https://code.google.com/archive/p/android-serialport-api/source/default/source>）：

```
/*打开串口之前调用下面的代码*/
File device = new
File("/dev/ttyS0");
/* check access permission */
if (!device.canRead() ||
!device.canWrite()) {
    try {
        /* Missing read/write
permission, trying to chmod the file
*/
        Process su;
        su =
Runtime.getRuntime().exec("/system/bin/su");
        String cmd = "chmod 666 "
+ device.getAbsolutePath() + "\n"
        \+ "exit\n";
```

```
su.getOutputStream().write(cmd.getBytes());

        if ((su.waitFor() != 0) ||
!device.canRead()

                ||
!device.canWrite()) {
                throw new
SecurityException();
        }
    } catch (Exception e)
        e.printStackTrace();
        throw new
SecurityException();
    }
}
```

七、错误码说明

错误码	错误码说明
0000	成功
0001	操作失败
1001	设备未打开
1002	参数无效
1003	无卡或无法寻到卡片
1101	磁条卡—设备处于等待刷卡状态
1102	磁条卡—设备处于停止刷卡状态
12A1	密码键盘-用户取消
12A2	密码键盘-输入超时
FFFF	操作失败(其他错误)

八、弃用列表

- 成员

[com.decard.NDKMethod.BasicOper.dc_BardCodeReceive](#) (byte[] msgSrc)

此接口因固件升级不推荐使用

- 成员

[com.decard.NDKMethod.BasicOper.dc_Config2DBarcodes](#) (int type, int value)

此接口因固件升级不推荐使用

- 成员 [com.decard.NDKMethod.BasicOper.dc_exit](#) ()

- 成员

[com.decard.NDKMethod.BasicOper.dc_get_id_raw_string](#) ()

此接口不推荐调用，直接调用

#dc_get_id_raw_info() 返回封装好的

[com.decard.entitys.IDCard](#) IDCard 对象

- 成员 [com.decard.NDKMethod.BasicOper.dc_open](#)

(String name, Context context, String param, int baudRate)

此接口已经废弃，直接调用::dc_open(String, Context, String, int) 参见#dc_open(String, Context, String, int) 此接口已经废弃，直接调用::dc_open(String, Context, String, int)

- 成员

[com.decard.NDKMethod.BasicOper.dc_SamAReadCardInfoString](#) (int type)

此接口不推荐调用，直接调用

#dc_SamAReadCardInfo() 返回封装好的
[com.decard.entitys.IDCard](#) IDCard 对象

- 成员

[com.decard.NDKMethod.BasicOper.dc_Scan2DBarcode](#) (int time_ms)

此接口因固件升级不推荐使用

- 成员

[com.decard.NDKMethod.BasicOper.dc_SetBarcodeCallback](#) (DCCallBack.BarcodeCallback
mCallback)

此接口因固件升级不推荐使用

- 成员

[com.decard.NDKMethod.EGovernment.EgAPP_SI_ReadSSCard](#) ()

内部封装接口，不建议直接进行调用，可调用
#EgAPP_SI_ReadSSCardInfo

九、接口说明

1、文件列表

1.1 BasicOper.java

```
/**
 * @param context 应用程序上下文
 * @return -1:usb初始化错误<br>
 * -2: 未寻找到符合指定vid,pid的设备
 * @brief 向系统申请USB权限。
 * @par 系统会弹出用户确认框，点击确定表示允许程序获取权限，取消表示不允许程序操作usb。
 */
public static int
dc_AUSB_ReqPermission(Context context);

/**
 * @param name 期望的端口类型 :
<br>
 * "LUSB" - libusb
Linux （平台上的usb驱动开发，设备需要获取root权限。）<br>
 * "AUSB" - Android
usb Api （需要app获取系统usb权限，设备无需root。）<br>
```

* "COM" - 串行通信
(需要设备串口路径。需要设备具有串口
通讯模块)

* "BT" - 经典蓝牙
(基于蓝牙RFCOMM协议进行通讯)

* "BLE" - 低功耗蓝
牙 (基于Smart Bluetooth, 需要设备
蓝牙模块4.0及以上版本。)

* @param context Android 系统中为应
用上下文

* @param param 参数

 当期望端口类型为 "BT" 或者
"BLE" 时, 此参数为设备蓝牙MAC地址。

*
 当期望端口类
型为 "AUSB" 或者 "LUSB" 时, 此参数可为空。

*
 当期望端口类
型为 "COM" 此参数为串口路径, 必填。

* @param baudRate
 当期望端口类
型为 "BT", "BLE", "LUSB"或"AUSB" 时, 此参
数无效, 固定传入0。

*
 当期望端口类
型为 "COM" 此参数为波特率, 必填。

* @return <0表示失败, >0表示成功

* @brief 打开设备端口。

* @par 说明: 建立设备的通讯并且分配相应
的资源, 大部分功能接口都需要在此过程后才能进
行。

* @attention 当期望的端口类型是AUSB
时，请务必先调用
#dc_AUSB_ReqPermission(Context) 向系统
申请USB权限，系统会弹出用户确认框，点击确定表
示允许程序获取权限，取消表示不允许程序操作
usb。

```
* @see
#dc_AUSB_ReqPermission(Context)
* @deprecated
*/
public static native int dc_open(String
name, Context context, String param,
int baudRate);

public static void
setDevicesUidListener(DevicesUidListene
r du1);

public static int dc_open_logan(String
name, Context context, String param,
int baudRate);

public static int dc_open_device(String
name, Context context, String param,
int baudRate);

/**
```

```

        * @return 成功: "0000|SDK版本信息"
<br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 获取SDK版本。
        * @par 说明: 获取SDK版本信息。
        */
public static native String
dc_getSdkver();

/**
        * @return 成功: "0000|设备固件版本信
息" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 获取设备版本。
        * @par 说明: 获取设备内部固件代码的版
本。
        */
public static native String
dc_getver();

/**
        * @return <0表示失败, ==0表示成功。
        * @brief 关闭设备。
        * @par 说明: 关闭设备的通讯和释放资源
        * @deprecated

```

```

    */
public static native int dc_exit();

public static int dc_exit_device();

/**
    * @param mSec 蜂鸣时间，单位为10毫秒。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 设备蜂鸣
    * @par 说明：设备蜂鸣器发出指定时间的蜂
鸣声
    */
public static native String dc_beep(int
mSec);

/**
    * @param code
    * @return 0 :成功； 其他值:失败
    * @brief 设置SDK调用系统语言环境。
    * @par 说明：此接口暂时仅影响接口返回结
果中的错误码提示语。
    * @n 0: 中文(默认)
    * @n 1: 英文
    */

```

```

public static native int
dc_setLanguageEnv(int code);

/**
    * @param ledNum    指示灯编号，0表示全
    部指示灯，1表示第一个指示灯，2表示第二个指示
    灯，以此类推。
    * @param openFlag 0-点亮，1-熄灭
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    * @brief 指示灯点亮/熄灭
    * @par 说明: 点亮/熄灭设备的指示灯。
    */
public static native String
dc_ctlled(int ledNum, int openFlag);
/**
    * @brief 15693卡透传指令。
    * @par 说明:
    * String sendData15693卡透传指令。
    * @param sendData 输入数据。
    * @return 成功: "0000|返回的数据"
    <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    */

```

```
public static native String
dc_Iso15693Communication(String
sendData);
/**
    * @param dispstr 要显示的字符串
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 液晶显示
    * @par 说明: 显示指定字符串到液晶屏幕上
    */
public static native String
dc_disp_str(String dispstr);
```

```
/**
    * @return 成功: "0000|0001" 表示已刷
磁卡。 <br>
    * 成功: "0000|1002" 表示卡座存在接触式
CPU卡。 <br>
    * 成功: "0000|1004" 表示卡座存在接触式
未知卡。 <br>
    * 成功: "0000|1008" 表示感应区存在身份
证。 <br>
    * 成功: "0000|1010" 表示感应区存在激活
前的Type A CPU卡或Type B CPU卡。 <br>
    * 成功: "0000|1020" 表示感应区存在激活
后的Type A CPU卡或Type B CPU卡。 <br>
```

- * 成功: "0000|1040" 表示感应区存在激活前的M1卡 。

- * 成功: "0000|1100" 表示感应区存在激活前的多张Type A CPU卡 。

- * 成功: "0000|1200" 表示感应区存在激活前的多张M1卡。

- * 成功: "0000|1400" 表示感应区同时存在激活前的Type A CPU卡和M1卡。

- * 成功: "0000|1800" 表示刷磁卡失败。

- * 成功: "0000|1000" 表示启动刷磁卡模式处于关闭状态。

- * 成功: "0000|2000" 表示启动刷磁卡模式处于开启状态。

- * 成功: "0000|4000 - 表示感应区身份证激活前后状态一致。

- * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode} 表示无卡或无法寻到卡片
- * @brief 检测多卡状态。
- * @par 说明: 检测设备中存在什么类型的卡片, 支持磁卡、非接、接触、身份证。
- * @attention 可以解析为一个无符号整数 (Big-Endian), 值为下面列出的一种或多种之和, 如果检测到多种卡型, 返回结果为各返回值之和
- * /

```
public static native String
dc_card_exist();

/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
 * @brief 射频复位
 * @par 说明: 复位设备的射频, 可以关闭,
关闭并且启动。
 */
public static native String dc_reset();

/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
 * @brief 射频关闭
 * @par 说明: 关闭设备的射频
 */
public static native String
dc_reset_close();

/**
 * @param mode 0x00表示对空闲卡进行操
作, 0x01表示对所有卡操作。
```

```
    * @return 成功: "0000|ATQA值" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 寻卡请求
    * @par 说明: 支持ISO 14443 Type A类
型卡片的寻卡请求。
    */
```

```
public static native String
dc_request(int mode);
```

```
/**
    * @param cnt 保留, 固定为0x00。
    * @return 成功: "0000|卡序列号(
HEX)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 防卡冲突
    * @par 说明: 支持ISO 14443 Type A类
型卡片的防卡冲突。
    */
```

```
public static native String
dc_anticoll(int cnt);
```

```
/**
    * @param cnt 保留, 固定为0x00。
```



```

        * @return 成功: "0000|卡序列号
(HEX)" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 第二级防卡冲突。
        * @par 说明: 支持ISO 14443 Type A类
型卡片的第二级防卡冲突。
    */
public static native String
dc_anticoll2(int cnt);

/**
    * @param snr 卡片序列号 (HEX)
    * @return 成功: "0000|SAK值(长度1字
节, HEX)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 选卡操作。
    * @par 说明: 通过指定序列号, 选取相应的
卡片。
    */
public static native String
dc_select(String snr); //unsigend int 在
java层只能用long来代替

/**
    * @param snr 卡片序列号 (HEX)

```

```

        * @return 成功: "0000|SAK值(长度1字节, HEX)" <br>
        * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}
        * @brief 第二级选卡操作。
        * @par 说明: 通过指定序列号, 选取相应的卡片。

    */
public static native String
dc_select2(String snr); // unsigned int
在java层只能用long来代替

/**
    * @param mode 0x00表示对空闲卡进行操作, 0x01表示对所有卡操作。
    * @return 成功: "0000|卡片序列号(16进制字符串表示)" <br>
    * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}
    * @brief 寻卡请求、防卡冲突、选卡操作。
    * @par 说明: 内部包含了 #dc_request #dc_anticolll #dc_select 的功能。
    * @see #dc_request(int)
    #dc_anticolll(int) #dc_select(int)
    */

```

```

public static native String
dc_card_hex(int mode);

/**
    * @param mode 0x00表示对空闲卡进行操作，0x01表示对所有卡操作。
    * @return 成功: "0000|卡片序列号(16进制字符串表示)" <br>
    * 失败: "错误码|错误提示信息" {@link com.deccard.driver.references.ErrorCode}
    * @brief 寻卡请求、防卡冲突、选卡操作。
    * @par 说明: 内部包含了 #dc_request #dc_antico11 #dc_select #dc_antico112 #dc_select2 的功能。
    * @see #dc_request(int) #dc_antico11(int) #dc_select(int) #dc_antico112 #dc_select2
    */
public static native String
dc_card_n_hex(int mode);

/**
    * @param AFI 应用族标识符 固定0x00
    * @param N 时间槽编号 固定0x00
    * @return 成功: "0000|(返回的ATQB值, 16进制字符串表示)" <br>

```

```

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 寻卡请求。
        * @par 说明: 支持ISO 14443 Type B类
型卡片的寻卡请求。
    */
public static native String
dc_request_b_hex(byte AFI, byte N);

/**
    * @param PUPI 唯一PICC标识符 , 由
dc_request_b_hex返回的ATQB。
    * @param CID  CID 信道号 固定0x00。
    * @return 成功: "0000|(返回的ATQB值,
16进制字符串表示)" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 激活卡片。
    * @par 说明: 支持ISO 14443 Type B类
型卡片的激活。
    */
public static native String
dc_attrib(String PUPI, int CID);

/**

```

```

        * @return 成功: "0000|(返回的激活信息, 16进制字符串表示)" <br>
        * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}
        * @brief 寻Type B卡并激活。
        * @par 说明: 对Type B卡进行寻卡和激活。

    */
    public static native String
    dc_card_b_hex();

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}
    * @brief 终止卡操作
    * @par 说明: 使卡片进入终止状态, 此时必须把卡移出感应区后再次放入感应区才能寻到这张卡。

    */
    public static native String dc_halt();

/**
    * @param mode 模式, 同
    #dc_authentication

```

```

        * @param secNum 对应装载的扇区号
        * @param key      写入设备中的卡密码，固
        定为6个字节，格式HEX
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decard.driver.references.ErrorCode}
        * @brief 装载设备密码
        * @par 说明: 装载M1卡密码到设备内部，装
        载后可以在特定时候用 ::dc_authentication 来
        验证M1卡密码。注意: 由于多次装载设备密码可能会
        受硬件存储寿命限制，此接口只用于密码相对固定，
        装载次数少的场合。

        * @see #dc_authentication(int,
        int)
        */
public static native String
dc_load_key_hex(int mode, int secNum,
String key);

/**
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decard.driver.references.ErrorCode}
        * @brief 验证M1卡密码。
        * @par 说明:
        * 使用传入的密码来验证M1卡密码。
        * @param[in] icdev 设备标识符。

```

```

        * @param[in] _Mode 模式，0x00表示验证A密码，0x04表示验证B密码。

        * @param[in] _Addr 要验证密码的扇区号。

        * @param[in] passbuff 密码，固定为12字符。

        */
public static native String
dc_authentication_pass(int mode, int
secNum, String password);

/**
        * @param mode    <br> 0x00 - 表示用
        设备内部装载的第0套A密码来验证当前选取卡片的A
        密码。

        *                <br> 0x01 - 表示用
        设备内部装载的第1套A密码来验证当前选取卡片的A
        密码。

        *                <br> 0x02 - 表示用
        设备内部装载的第2套A密码来验证当前选取卡片的A
        密码。

        *                <br> 0x04 - 表示用
        设备内部装载的第0套B密码来验证当前选取卡片的B
        密码。

        *                <br> 0x05 - 表示用
        设备内部装载的第1套B密码来验证当前选取卡片的B
        密码。

```

* **
 0x06** - 表示用设备内部装载的第**2套B**密码来验证当前选取卡片的**B**密码。

* **@param secNum** 要验证密码的扇区号

* **@return** 成功: "0000|" **
**

* 失败: "错误码|错误提示信息" {@link com.deccard.driver.references.ErrorCode}

* **@brief** 验证M1卡密码

* **@par** 说明: 使用设备内部装载的密码来验证M1卡密码。

***/**

```
public static native String  
dc_authentication(int mode, int  
secNum);
```

/**

* **@param secNum** 扇区号

* **@param keyA** 密码A, 固定为6个字节

* **@param b0** 块0控制字 (当一扇区有16块时, 对应为块0~4的控制字), 低3位 (D2D1D0) 对应C10、C20、C30。(HEX)

* **@param b1** 块1控制字 (当一扇区有16块时, 对应为块5~9的控制字), 低3位 (D2D1D0) 对应C11、C21、C31。(HEX)


```

    * @param b2      块2控制字（当一扇区有
16块时，对应为块10~14的控制字），低3位
（D2D1D0）对应C12、C22、C32。（HEX）

    * @param b3      块3控制字（当一扇区有
16块时，对应为块15的控制字），低3位（D2D1D0）
对应C13、C23、C33。（HEX）

    * @param bk      保留，固定为00。
（HEX）

    * @param keyB    密码B，固定为6个字
节。

    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}

    * @brief M1卡密码配置块操作。

    * @par 说明：修改M1卡密码配置块数据，M1
卡密码配置块也就是每个扇区的最后一块，包含密码
A、控制字节、密码B数据。

    */
//      public static native String
dc_changeb3_hex(int secNum,String
keyA,String b0,String b1,String
b2,String b3,String bk,String keyB);
public static native String
dc_changeb3_hex(int secNum, String
keyA, byte b0, byte b1, byte b2, byte
b3, byte bk, String keyB);

```

```
/**
 * @param index 块号:<br>
 *                M1卡 - S50块地址
 *                (0~63), S70块地址 (0~255)。<br>
 *                ML卡 - 页地址
 *                (0~11)。
 * @return 成功: "0000|块数据 (--长度
16字节, 16进制字符串表示, 真实数据可能小于16个
字节)" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decaddriver.references.ErrorCode}
 * @brief 读卡数据。
 * @par 读取卡内数据, 对于M1卡, 一次读取
一个块的数据, 为16个字节; 对于ML卡, 一次读取相
同属性的两页, 为8个字节。
 */
public static native String
dc_read_hex(int index);
```

```
/**
 * @param blockNum 块号:<br>
 *                M1卡 - S50块地址
 *                (0~63), S70块地址 (0~255)。<br>
 *                ML卡 - 页地址
 *                (0~11)。
```

```

        * @param valueHex 块数据 长度固定为16
        字节，16进制字符串形式
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decadd.driver.references.ErrorCode}
        * @brief 写卡数据。
        * @par 说明: 写入数据到卡片内，对于M1
        卡，一次必须写入一个块的数据，为16个字节；对于
        ML卡，一次必须写入一个页的数据，为4个字节。
        */
public static native String
dc_write_hex(int blockNum, String
valueHex);

/**
        * @param value 初始化数值（非负）
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decadd.driver.references.ErrorCode}
        * @brief ML卡值初始化。
        * @par 说明: 用于操作ML卡，初始化卡内容
        为指定数值。
        */
public static native String
dc_initval_ml(int value);

```

```
/**
 * @return 成功: "0000|数值" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
 * @brief ML卡读值。
 * @par 说明: 用于读取ML卡数值。
 */
```

```
public static native String
dc_readval_m1();
```

```
/**
 * @param value 要减少的数值
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
 * @brief ML卡减值。
 * @par 说明: 用于操作ML卡值, 使值减少指
定数值。
 */
```

```
public static native String
dc_decrement_m1(int value);
```

```
/**
 * @param key key 密码, 固定为16个字
节。
 * @return 成功: "0000|" <br>
```

```

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 验证Mifare Ultralight C卡
密码。

        * @par 说明: 使用传入的密码来验证
Mifare Ultralight C卡密码。

    */
public static native String
dc_auth_ulc_hex(String key);

/**
        * @param newKey key 密码, 固定为16个
字节。
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 修改Mifare Ultralight C卡
密码。

        * @par 说明: 修改Mifare Ultralight C
卡的密码。

    */
public static native String
dc_changekey_ulc_hex(String newKey);

/**
        * @param blockAddr 要回传的块地址
        * @return 成功: "0000|" <br>

```

```

    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief M1卡值块回传
    * @par 说明: 用于把指定的值块内容暂存,
后续可以使用 dc_transfer 将暂存值块内容传递
到另一块中, 实现块与块之间数值传送。
    * @see #dc_transfer(int)
    */
public static native String
dc_restore(int blockAddr);

/**
    * @param blockAddr 要传递的块地址
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief M1卡值块传递
    * @par 说明: 用于操作M1卡值块, 使块值增
加指定数值 :: dc_restore
    * @see #dc_restore(int)
    */
public static native String
dc_transfer(int blockAddr);

/**
    * @param blockAddr 块地址
    * @param value      要增加的数值

```

```

    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief M1卡值块加值
    * @par 说明: 用于操作M1卡值块, 使块值增
加指定数值。
    */
public static native String
dc_increment(int blockAddr, int value);

/**
    * @param blockAddr 块地址
    * @param value      要减少的数值
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief M1卡值块减值
    * @par 说明: 用于操作M1卡值块, 使块值减
少指定数值
    */
public static native String
dc_decrement(int blockAddr, int value);

/**
    * @param blockAddr 块地址
    * @param value      初始化数值
    * @return 成功: "0000|" <br>

```

```

    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief M1卡值块初始化
    * @par 说明: 用于操作M1卡, 使得指定块为
值块, 并且初始化此值块为指定数值。
    */
public static native String
dc_initval(int blockAddr, int value);

/**
    * @param blockAddr 要读取的块地址
    * @return 成功: "0000|(读取的值)"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief M1卡值块读值
    * @par 说明: 用于读取M1卡值块数值
    */
public static native String
dc_readval(int blockAddr);

/**
    * @param sn 卡座编号
    *
    * <br> 0 - 附卡座/接触式
CPU1卡座。

```



```

*          <br> 1 - 接触式CPU2卡
座。
*          <br> 2 - SAM1卡座。
*          <br> 3 - SAM2卡座。
*          <br> 4 - SAM3卡座。
*          <br> 5 - SAM4卡座。
*          <br> 6 - SAM5卡座。
*          <br> 7 - SAM6卡座。
*          <br> 8 - SAM7卡座。
*          <br> 9 - SAM8卡座。
* @return 成功: "0000|" <br>
* 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
* @brief 设置当前接触式卡座。
* @par 说明: 设置当前接触式卡座为指定卡
座, 用于多卡座切换卡操
*/
public static native String
dc_setcpu(int sn);

/**
* @return 成功: "0000|" <br>
* 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
* @brief 设置接触式CPU卡参数。
* @par 说明:

```

* 此接口可控制接触式CPU卡操作的行为，当不掉用时，设备将使用默认方式进行接触式CPU卡操作。

* @param[in] sn 卡座编号，同 ::dc_setcpu 的 @a _Byte 。

* @param[in] cpupro 卡协议编号，0x00表示T0，0x01表示T1，默认为0x00。

* @param[in] cpuetu 卡复位波特率编号，0x5C表示9600，0x14表示38400。

*/

```
public static native String  
dc_setcpupara(int sn, int cpupro, int  
cpuetu);
```

/**

* @return 成功: "0000|" 卡片存在

* 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}

* @brief 检测接触式卡存在。

* @par 说明: 检测接触式卡片是否存在于
当前卡座中。

*/

```
public static native String  
dc_card_status();
```

```
/**
    * @return 成功: "0000|" 无卡
    * "0001|" 存在身份证
    * "0002|" 存在Type A CPU卡
    * "0003|" 存在Type B CPU卡
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 身份证、Type A CPU卡、Type
B CPU卡检测。
    * @par 说明:
    * 检测感应区是否存在身份证、Type A CPU
卡、Type B CPU卡。
    */
```

```
public static native String
dc_typeab_card_status();
```

```
/**
    * @return 成功: "0000|复位信息 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 接触式CPU卡复位。
    * @par 说明: 对当前卡座CPU卡进行复位操
作, 此复位为冷复位。
    */
```

```
public static native String
dc_cpureset_hex();
```

```
/**
    * @param apduHex 发送的APDU数据。(请
    应用层检查输入的数据是否是合法16进制字符串)
    * @return 成功: "0000|响应APDU ( HEX
    )" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    * @brief 接触式CPU卡指令交互。
    * @par 说明: 对当前卡座CPU卡进行指令交
    互操作, 注意此接口已封装卡协议部分。
    * @attention 此接口c语言层使用的数据类
    型为 unsigned char 范围(0-255), apdu长度
    超过255请使用 #dc_cpuapduInt_hex 接口。
    */
public static native String
dc_cpuapdu_hex(String apduHex);

/**
    * @brief 接触式CPU卡指令交互。
    * @par 说明: 对当前卡座CPU卡进行指
    令交互操作, 注意此接口不封装卡协议部分。
    * @param apduHex 发送的APDU数据。(请
    应用层检查输入的数据是否是合法16进制字符串)
```

* ::dc_cpuapdusourceEXT 的HEX形式接口，参数 @a sendbuffer @a databuffer 为HEX格式。

*/

```
public static native String  
dc_cpuapdusourceEXT_hex(String  
apduHex);
```

/**

* @param apduHex 发送的APDU数据。(请应用层检查输入的数据是否是合法16进制字符串)

* @return 成功: "0000|响应APDU (HEX)"

* 失败: "错误码|错误提示信息" {@link com.deccard.driver.references.ErrorCode}

* @brief 接触式CPU卡指令交互。

* @par 说明: 对当前卡座CPU卡进行指令交互操作，注意此接口已封装卡协议部分。

* @attention 此接口c语言层使用的数据类型为 unsigned int ， apdu长度超过255请使用此接口。

*/

```
public static native String  
dc_cpuapduInt_hex(String apduHex);
```

/**

* @param apduHex 发送的APDU数据。(请应用层检查输入的数据是否是合法16进制字符串)。

* @return 成功: "0000|响应APDU (HEX)"

* 失败: "错误码|错误提示信息" {@link com.deccard.driver.references.ErrorCode}

* @brief 接触式CPU卡指令交互。

* @par 说明: 对当前卡座CPU卡进行指令交互操作, 注意此接口已封装卡协议部分, 内部处理了SW1为0x61和0x6C的情况。

*/

```
public static native String  
dc_cpuapduresponInt_hex(String  
apduHex);
```

/**

* @return 成功: "0000|"

* 失败: "错误码|错误提示信息" {@link com.deccard.driver.references.ErrorCode}

* @brief 接触式CPU卡下电。

* @par 说明: 对当前卡座CPU卡进行下电操作。

*/

```
public static native String  
dc_cpudown();
```

```
/**
    * @return 成功: "0000|卡类型 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * <br> 0x00 - 无卡。
    * <br> 0x11 - Type A CPU Card。
    * <br> 0x13 - Type A Mifare S50。
    * <br> 0x14 - Type A Mifare S70。
    * <br> 0x15 - Type A Mifare
Ultralight。
    * <br> 0x21 - Type B CPU Card。
    * <br> 0x22 - Type B 存储卡。
    * @brief 获取非接触卡类型。
    * @par 说明: 获取感应区中存在的非接触卡
类型。
    */
public static native String
dc_RfGetCardType();
```

```
/**
    * @return 成功: "0000|复位信息 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 非接触式CPU卡复位。
```

```

        * @par 说明：对感应区CPU卡进行复位操作。

        */
public static native String
dc_pro_resethex();

/**
    * @return 成功: "0000|复位信息 ( HEX )" <br>
    * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}
    * @brief 非接触式CPU卡复位。
    * @par 说明：对感应区CPU卡进行复位操作。

    */
public static native String
dc_pro_resetInt_hex();

/**
    * @param apduHex 发送的APDU数据。(请应用层检查输入的数据是否是合法16进制字符串)
    * @param timeout 超时值，此值只在部分设备的底层使用，单位为250毫秒，一般调用建议值为7。

```



```

        * @return 成功: "0000|响应APDU ( HEX
)" <br>
        * 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}
        * @brief 非接触式CPU卡指令交互。
        * @par 说明: 对感应区CPU卡进行指令交互
操作, 注意此接口已封装卡协议部分。
        * @attention 此接口c语言层使用的数据类型
为 unsigned char 范围 (0-255), apdu长度
超过255请使用 #dc_procommandInt_hex 接口。
    */
public static native String
dc_pro_commandhex(String apduHex, int
timeout);

/**
        * @param apduHex 发送的APDU数据。(请
应用层检查输入的数据是否是合法16进制字符串)。
        * @param timeout 超时值, 此值只在部分
设备的底层使用, 单位为250毫秒, 一般调用建议值
为7。
        * @return 成功: "0000|响应APDU ( HEX
)" <br>
        * 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}
        * @brief 非接触式CPU卡指令交互。

```

* @par 说明： 对当前卡座CPU卡进行指令交互操作，注意此接口已封装卡协议部分。

* @attention 此接口c语言层使用的数据类型为 unsigned int ， apdu长度超过255请使用此接口。

*/

```
public static native String  
dc_procommandInt_hex(String apduHex,  
int timeout);
```

/**

* @return 成功: "0000|"

* 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}

* @brief 终止非接触式CPU卡操作。

* @par 说明： 使非接触式CPU卡进入终止状态，此时必须把卡移出感应区后再次放入感应区才能寻到这张卡。

*/

```
public static native String  
dc_pro_halt();
```

/**

* @return 成功: "0000|"


```

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 启动读磁条卡。
        * @par 说明: 在设备进入监测刷磁条的状态
后, 只有刷完磁条卡或调用此接口才能使得设备退出
这种状态。
        */
public static native String
dc_startreadmag();

/**
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
        * @brief 停止读磁条卡。
        * @par 说明: 在设备进入监测刷磁条的状态
后, 只有刷完磁条卡或调用此接口才能使得设备退出
这种状态。
        */
public static native String
dc_stopreadmag();

/**
        * @return 成功: "0000|磁道1数据
(HEX)|磁道2数据(HEX)|磁道3数据(HEX)" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}

```

```

        * 失败: "1002|等待刷卡"
        * 失败: "1003|设备处于停止刷卡状态"
        * @brief 获取磁条卡数据。
        * @par 说明: 读取磁条卡的数据, 支持1、
2、3轨道。
    */
public static native String
dc_readmag();

/**
    * @param timeout 接口超时时间, 单位为
秒。
    * @return 成功: "0000|磁道1数据
(HEX)|磁道2数据(HEX)|磁道3数据(HEX)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}
    * 失败: "1002|等待刷卡"
    * 失败: "1003|设备处于停止刷卡状态"
    * @brief 获取磁条卡数据。
    * @par 说明: 读取磁条卡的数据, 支持1、
2、3轨道。
    */
public static native String
dc_readmagcardall(int timeout);

```

```
public static native int
dc_wlt2bmp(byte[] wlt, byte[] bmp);

/**
    * @return 成功: "0000|文字信息|相片信息|指纹信息|追加住址信息
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 读身份证原始数据, 此接口返回结果通过字符串拼接而成, 中间使用“|”分割, 请直接调用 #dc_get_i_d_raw_info() 获取解析后的对象更加方便。。
    * @par 说明: 读取身份证的原始信息数据。
    * @param[in] type 类型。
    * @n 1 - 读取文字信息、相片信息和指纹信息。
    * @n 2 - 读取追加住址信息。
    * @n 3 - 读取文字信息、相片信息、指纹信息和追加住址信息。
    * @deprecated 此接口不推荐调用, 直接调用 #dc_SamAReadCardInfo() 返回封装好的 {@link
com.decard.driver.entitys.IDCard} IDCard 对象
    */
private static native String
dc_SamAReadCardInfoString(int type);
```

```
/**
 * @param type 类型。
 * @return 成功: IDCard 对象 <br>
 * 失败: null
 * @brief 读取身份证信息（支持指纹读取）
 * @n 1 - 读取文字信息、相片信息和指纹信息。
 *
 * @n 2 - 读取追加住址信息。
 * @n 3 - 读取文字信息、相片信息、指纹信息和追加住址信息。
 */
public static IDCard
dc_SamAReadCardInfo(int type);

/**
 * @return 成功: "0000|身份证信息"
<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 读身份证。
 * @par 说明:
 * 读取身份证的原始信息数据，此接口使用硬件加速，需固件支持。
 * @param[in] mode 模式，为0x00表示定制卡（内部使用），为0x01表示身份证卡。
```

```

    * @param[in] type 类型，为0x00表示内部使用，否则值为下面列出的一种或多种之和。
    * @n 0x01 - 读取文字信息。
    * @n 0x02 - 读取相片信息。
    * @n 0x04 - 读取指纹信息。
    * @n 0x08 - 读取追加住址信息。
    * @param[out] text_len 返回文字信息的长度。
    * @param[out] text 返回的文字信息，请至少分配256个字节。
    * @param[out] photo_len 返回相片信息的长度。
    * @param[out] photo 返回的相片信息，请至少分配1024个字节。
    * @param[out] fingerprint_len 返回指纹信息的长度。
    * @param[out] fingerprint 返回的指纹信息，请至少分配1024个字节。
    * @param[out] extra_len 返回追加住址信息的长度。
    * @param[out] extra 返回的追加住址信息，请至少分配70个字节。
    */
public static native String
dc_IdCardReadCardInfo(int mode, int
type);
```

```
/**
 * @param type 类型。
 * @return 成功: IDCard 对象 <br>
 * 失败: null
 * @brief 读取身份证信息（支持指纹读取）
 * @n 7 - 读取文字信息、相片信息和指纹信息。
 */
public static IDCard
dc_IdCardReadCardInfo(int type);

/**
 * @return 成功: 0000|姓名|性别|民族|
生日|地址|ID号码|签发机关|有效期开始|有效期结束 <br>
 * 失败: null
 * @brief 读身份证原始数据，此接口返回结果通过字符串拼接而成，中间使用“|”分割，请直接调用 #dc_get_i_d_raw_info() 获取解析后的对象更加方便。
 * @par 说明：读取身份证模块返回的原始数据，包含基础文字和相片内容。
 * @see #dc_get_i_d_raw_info()
```



```
        * @deprecated 此接口不推荐调用，直接调用 #dc_get_i_d_raw_info() 返回封装好的 {@link com.decard.driver.entitys.IDCard} IDCard 对象
    */
    private static native String
    dc_get_i_d_raw_string();

    /**
        * @return 成功: IDCard 对象 <br>
        * 失败: null
        * @brief 读身份证原始数据（不支持指纹读取）。
        * @par 说明：读取身份证模块返回的原始数据，包含基础文字和相片内容。
        * @see
        com.decard.driver.entitys.IDCard
    */
    public static IDCard
    dc_get_i_d_raw_info();

    /**
        * @return 成功: "0000|返回的 UID(HEX)" <br>
        * 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}
```

```

        * @brief 获取身份证UID。
        * @par 说明：获取身份证的唯一标识，长度为8个字节。
        */
public static native String
dc_getuid_id();

/**
    * @return 成功: "0000|指纹数据(HEX)"
<br>
    * 失败: "错误码|错误提示信息" {@link com.decord.driver.references.ErrorCode}
    * @brief 获取身份证指纹信息。
    * @par 说明：获取身份证指纹信息。
    */
public static native String
dc_getfingerprintdata();

/**
    * @return 成功: "0000|标识符" <br>
    * 成功: "0000|08" 表示存在4442卡。
<br>
    * 成功: "0000|09" 表示存在4428卡。
<br>
    * 成功: "0000|30" 表示存在T=0的CPU卡。 <br>

```

```

    * 成功: "0000|31" 表示存在T=1的CPU
卡。<br>
    * 成功: "0000|21" 表示存在24C01卡。
<br>
    * 成功: "0000|22" 表示存在24C02卡。
<br>
    * 成功: "0000|23" 表示存在24C04卡。
<br>
    * 成功: "0000|24" 表示存在24C08卡。
<br>
    * 成功: "0000|25" 表示存在24C16卡。
<br>
    * 成功: "0000|26" 表示存在24C64卡。
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
表示失败或不存在。
    * @brief 检测卡
    * @par 检测当前卡座中存在的卡类型。
    */
public static native String
dc_checkCard();

/**
    * @return 成功: "0000|" 表示存在
4442卡。<br>
```

```
        * 失败: "0001|" 表示失败或不存在。
<br>
        * @brief 检测4442卡。
        * @par 说明: 检测当前卡座中是否存在4442
卡。
        */
public static native String
dc_check_4442();

/**
        * @return 成功: "0000|" 表示存在
4428卡。<br>
        * 失败: "0001|" 表示失败或不存在。
<br>
        * @brief 检测4428卡。
        * @par 说明: 检测当前卡座中是否存在4428
卡。
        */
public static native String
dc_check_4428();

/**
        * @param passwordHex 密码(HEX), 固定
为2个字节。
        * @return 成功: "0000|" <br>
```

```
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 验证4428卡密码。
        * @par 说明: 使用传入的密码来验证4428卡
密码。
        */
public static native String
dc_verifypin_4428_hex(String
passwordHex);

/**
        * @return 成功: "0000|密码(HEX), 固定
为2个字节" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 读4428卡密码。
        * @par 说明: 读取4428卡的密码。
        */
public static native String
dc_readpin_4428_hex();

/**
        * @return 成功: "0000|密码(HEX), 固定
为2个字节" <br>
```

```
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 读4428卡密码计数。

        * @par 说明: 读取4428卡的密码计数, 此计
数值表示可以尝试验证密码的次数。

        */
public static native String
dc_readpincount_4428();

/**
        * @param newPasswordHex passwd 密码
(Hex), 固定为2个字节

        * @return 成功: "0000|" <br>

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 修改4428卡密码

        * @par 说明: 修改4428卡的密码

        */
public static native String
dc_changepin_4428_hex(String
newPasswordHex);

/**

        * @param offset 偏移量

        * @param length 读取数据长度
```

* @return 成功: "0000|数据(HEX)" ---
返回的数据, 数据中含有0x00字节的位置表示已经被
置保护

* 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}

* @brief 读4428卡保护位

*/

```
public static native String  
dc_readprotect_4428_hex(int offset, int  
length);
```

```
/**
```

* @param offset 偏移量

* @param length 写入数据长度

* @param dataHex 写入数据 (HEX) ---
数据中和卡内原有数据相同的字节位置将被置保护

* @return 成功: "0000|"

* 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}

* @brief 写4428卡保护位

* @par 对4428卡的保护区中指定位置进行置
保护

*/

```
public static native String
dc_writeprotect_4428_hex(int offset,
int length, String dataHex);

/**
 * @param offset 偏移量
 * @param length 读取数据长度
 * @return 成功: "0000|返回的数据
(HEX)"<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 读4428卡
 * @par 说明: 读取4428卡的数据
 */
public static native String
dc_read_4428_hex(int offset, int
length);

/**
 * @param offset 偏移量
 * @param length 写入数据长度
 * @param dataHex 写入的数据 (HEX)
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
```



```

        * @brief 写4428卡
        * @par 说明： 写入数据到4428卡中
        */
public static native String
dc_write_4428_hex(int offset, int
length, String dataHex);

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 4428卡下电
    * @par 说明： 对4428卡进行下电操作
    */
public static native String
dc_down_4428();

/**
    * @param passwordHex 密码(HEX)，固定
为3个字节。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 验证4442卡密码。

```

```
    * @par 说明：使用传入的密码来验证4442卡
    密码。

    */
public static native String
dc_verifypin_4442_hex(String
passwordHex);

/**
    * @return 成功: "0000|密码(HEX)，固定
    为3个字节" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.deccard.driver.references.ErrorCode}
    <br>
    * @brief 读4442卡密码。
    * @par 说明：读取4442卡的密码。
    */
public static native String
dc_readpin_4442_hex();

/**
    * @return 成功: "0000|密码(HEX)，固定
    为2个字节" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.deccard.driver.references.ErrorCode}
    <br>
    * @brief 读4442卡密码计数。
```

```

        * @par 说明：读取4442卡的密码计数，此计
数值表示可以尝试验证密码的次数。

    */
public static native String
dc_readpincount_4442();

/**
    * @param newPasswordHex passwd 密码
    (HEX)，固定为3个字节
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 修改4442卡密码
    * @par 说明：修改4442卡的密码
    */
public static native String
dc_changepin_4442_hex(String
newPasswordHex);

/**
    * @param offset 偏移量
    * @param length 读取数据长度
    * @return 成功: "0000|数据(HEX)" ---
返回的数据，数据中含有0x00字节的位置表示已经被
置保护 <br>

```

```

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 读4442卡保护位
        */
public static native String
dc_readprotect_4442_hex(int offset, int
length);

/**
        * @param offset 偏移量
        * @param length 写入数据长度
        * @param dataHex 写入数据 (HEX) ---
数据中和卡内原有数据相同的字节位置将被置保护
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 写4442卡保护位
        * @par 对4442卡的保护区中指定位置进行置
保护
        */
public static native String
dc_writeprotect_4442_hex(int offset,
int length, String dataHex);

/**

```

```

    * @param offset 偏移量
    * @param length 读取数据长度
    * @return 成功: "0000|返回的数据
(HEX)"<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 读4442卡
    * @par 说明: 读取4442卡的数据
    */
public static native String
dc_read_4442_hex(int offset, int
length);

/**
    * @param offset 偏移量
    * @param length 写入数据长度
    * @param dataHex 写入的数据 (HEX)
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 写4442卡
    * @par 说明: 写入数据到4442卡中
    */

```

```

public static native String
dc_write_4442_hex(int offset, int
length, String dataHex);

/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 4442卡下电
 * @par 说明: 对4442卡进行下电操作
 */
public static native String
dc_down_4442();

/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 唤醒设备。
 * @par 说明: 唤醒休眠中的设备, 此接口用
于向设备发出唤醒请求, 执行成功仅仅表示成功发出
唤醒请求, 并不表示设备已经唤醒。
 */
public static native String
dc_wakeDevice();

```

```
/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 休眠设备。
 * @par 说明: 休眠设备, 此接口用于向设备
发出休眠请求, 执行成功仅仅表示成功发出休眠请
求, 并不表示设备已经进入休眠状态。
 */
public static native String
dc_RestDevice();
```

```
/**
 * @return 成功: "0000|电池百分比
(0~100)" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取电池电量。
 * @par 说明: 获取电池电量。
 */
public static native String
dc_GetBatteryValue();
```

```
/**
 * @param timeout 设备超时值，单位为
秒。
 * @return 成功: "0000|返回的按键值"
<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取单个按键值。
 * @par 说明: 获取设备键盘的按键值，每次
调用仅能获取单个按键值。
 */
```

```
public static native String
dc_pass_key_value(int timeout);
```

```
/**
 * @param timeout 设备超时值，单位为
秒。
 * @return 成功: "0000|返回的按键值"
<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取多个按键值。
```


* @par 说明：获取设备键盘的按键值，每次调用可以获取多个按键值。

*/

```
public static native String  
dc_pass_key_value2(int timeout);
```

/**

* @param timeout 设备超时值，单位为秒。

* @return 成功: "0000|"

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 进入读键盘模式。

* @par 说明：使设备进入读取键盘的状态，接口本身会直接返回，而不会等待按键完成或设备超时。

* @attention 接口本身会直接返回，而不会等待按键完成或设备超时，后面通过{@link #dc_PassGet()}来获取键盘输入数据。

* @see #dc_PassGet()

*/

```
public static native String  
dc_PassIn(int timeout);
```

```
/**
 * @return 成功: "0000| 返回的按键数据"
<br>
 * 失败: "错误码| 错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取键盘数据。
 * @par 说明: 获取设备已经读取到的键盘数
据。
 * @attention 使用前请调用
#dc_PassGet() 进入读键盘模式
 * @see #dc_PassGet()
 */
public static native String
dc_PassGet();
```

```
/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码| 错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 退出读键盘模式。
 * @par 说明: 在设备进入读键盘模式后, 设
备会一直处于接收键盘输入的状态, 这时是只有按键
完成、设备超时或调用此接口才能使得设备退出这种
状态。
```

```

    */
public static native String
dc_PassCancel();

/**
    * @param timeout 设备超时值，单位为
    秒。
    * @return 成功: "0000|返回的按键数据"
    <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    <br>
    * @brief 键盘密码输入
    * @par 说明: 获取键盘按键，用于密码输
    入，不同的键盘装置可能有不同的效果，如不同的提
    示音或不同的显示。
    * @attention 此接口相当于同步效果。
    */
public static native String
dc_GetInputPass(int timeout);

/**
    * @param keyID 密钥组号，从0x00开始编
    号。
    * @return 成功: "0000|" <br>

```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 设置当前密钥组。
```

```
    * @par 说明: 设置密码键盘当前使用的密钥  
组。
```

```
    */
```

```
public static native String  
dc_SetKbKeyID(int keyID);
```

```
/**
```

```
    * @param flag          标记。
```

```
    * @param mode          模式, 0x00表示单  
DES密钥, 0x01表示三DES密钥, 0x02表示SM4密  
钥, 0x03表示24字节三DES密钥。
```

```
    * @param keyDataHex 密钥数据 (HEX)。
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 设置密码键盘密钥。
```

```
    * @par 说明: 对密码键盘内的密钥进行设置  
操作。
```

```
    * @n 0x00 - 表示装载主密钥, 明文装载。
```

```
    * @n 0x01 - 表示装载PIN密钥, 密文装载  
(用主密钥加密)。
```

* @n 0x02 - 表示装载MAC密钥，密文装载（用主密钥加密）。

* @n 0x03 - 表示装载TDK密钥，密文装载（用主密钥加密）。

* @n 0xF0 - 表示清空主密钥、PIN密钥、MAC密钥、TDK密钥。

* @n 0xF1 - 表示清空PIN密钥。

* @n 0xF2 - 表示清空MAC密钥。

* @n 0xF3 - 表示清空TDK密钥。

*/

```
public static native String  
dc_InstallKbKey(int flag, int mode,  
String keyDataHex);
```

```
/**
```

* @param timeout 设备超时值，单位为秒。

* @param cardNo 卡号。

* @return 成功: "0000|返回的加密数据"

* 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}

* @brief 获取加密按键数据。

* @par 说明: 从密码键盘获取ANSI X9.8
PINBLOCK加密的按键数据。

```

    */
public static native String
dc_GetEnPass(int timeout, String
cardNo);

/**
    * @param key          固定8个字节密钥
    (16个长度字符串HEX)。
    * @param srcDataHex  要做加密/解密的数据
    (8个字节,16个长度字符串,HEX格式)。
    * @param mode        模式, 0-解密, 1-
    加密。
    * @return 成功: "0000|8个字节运算后的
    数据(16个长度字符串,HEX格式)" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.deccard.driver.references.ErrorCode}
    <br>
    * @brief 单DES加解密。
    * @par 说明: 使用单DES算法对数据进行加
    密/解密。
    */
public static native String
dc_des_hex(String key, int mode, String
srcDataHex);

```

```

/**
    * @param key          固定16个字节密钥
    (32个长度字符串,HEX格式)。
    * @param srcDataHex  要做加密/解密的数据
    (8个字节,16个长度字符串,HEX格式)。
    * @param mode        模式, 0-解密, 1-
    加密。
    * @return 成功: "0000|8个字节运算后的
    数据(16个长度字符串,HEX格式)"  <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    <br>
    * @brief 三DES加解密。
    * @par 说明: 使用三DES算法对数据进行加
    密/解密。
    */
public static native String
dc_tripledes_hex(String key, int mode,
String srcDataHex);

/**
    * @param keyType      密钥类型,
    0x01表示PIN密钥, 0x02表示MAC密钥, 0x03表示
    TDK密钥。
    * @param enMode       运算模式,
    0x00表示加密, 0x01表示解密。

```

```

        * @param sourceDataHex 传入数据，固
        定为8个字节。

        * @return 成功: "0000|运算后的数据,固
        定为8个字节(16个长度字符串,HEX格式)" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decadd.driver.references.ErrorCode}
        <br>

        * @brief 用工作密钥加解密。

        * @par 使用密码键盘的工作密钥进行数据加
        解密操作，仅支持单DES、三DES和24字节三DES算
        法，算法由设置密钥时确定，请参考
        #dc_InstallKbKey(int, int, String)。

        * @see #dc_InstallKbKey(int, int,
        String)
        */
public static native String
dc_CreatDesData(int keyType, int
enMode, String sourceDataHex);

/**
        * @param keyType          密钥类型，
        0x01表示PIN密钥，0x02表示MAC密钥，0x03表示
        TDK密钥。

        * @param enMode          运算模式，
        0x00表示加密，0x01表示解密。

```



```

        * @param sourceDataHex 传入数据，固
        定为16个字节。

        * @return 成功: "0000|运算后的数据,固
        定为16个字节(32个长度字符串,HEX格式)" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decord.driver.references.ErrorCode}
        <br>

        * @brief 用工作密钥加解密。

        * @par 说明: 使用密码键盘的工作密钥进行
        数据加解密操作, 仅支持SM4算法, 算法由设置密钥
        时确定, 请参考 #dc_InstallKbKey(int, int,
        String) 。

        * @see #dc_InstallKbKey(int, int,
        String)
        */
public static native String
dc_CreatSm4Data(int keyType, int
enMode, String sourceDataHex);

/**
        * @param keyType 密钥类型,
        0x01表示PIN密钥, 0x02表示MAC密钥, 0x03表示
        TDK密钥。

        * @param enMode 运算模式,
        0x00表示加密, 0x01表示解密。

        * @param sourceDataHex 传入数据。

```

```

        * @return 成功: "0000|运算后的数据
(HEX)" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 用工作密钥加解密。
        * @par 说明: 使用密码键盘的工作密钥进行
数据加解密操作, 仅支持单DES ECB和三DES ECB算
法, 内部会调用 ::dc_CreatDesData 。
        * @see #dc_CreatDesData(int, int,
String)
        */
public static native String
dc_CreatDesDataECB(int keyType, int
enMode, String sourceDataHex);

/**
        * @param keyType          密钥类型,
0x01表示PIN密钥, 0x02表示MAC密钥, 0x03表示
TDK密钥。
        * @param enMode          运算模式,
0x00表示加密, 0x01表示解密。
        * @param sourceDataHex 传入数据。
        * @return 成功: "0000|运算后的数据
(HEX)" <br>

```

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 用工作密钥加解密。

* @par 说明: 使用密码键盘的工作密钥进行数据加解密操作, 仅支持单DES ECB和三DES ECB算法, 内部会调用 ::dc_CreatDesData 。

* @see #dc_CreatDesData(int, int, String)

*/

```
public static native String  
dc_CreatSm4DataECB(int keyType, int  
enMode, String sourceDataHex);
```

```
/**
```

* @param keyType 密钥类型,
0x01表示PIN密钥, 0x02表示MAC密钥, 0x03表示
TDK密钥。

* @param enMode enmode 运算
模式, 0x00或0x01表示用DES POS ECB算MAC,

* @param sourceDataHex 传入数据。

* @return 成功: "0000|运算后的数据
(HEX)"

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}


```

        * @brief 用工作密钥算MAC。
        * @par 说明：使用密码键盘的工作密钥进行
        数据算MAC操作，内部会调用
        #dc_CreatDesData(int, int, String)
        #dc_CreatSm4Data(int, int, String) 。
        * @n 0x02表示用SM4算MAC
        * @n 0x03表示用DES X9.9算MAC
        * @n 0x04表示用三DES X9.19算MAC
        * @n 0x05表示用三DES POS ECB算MAC
        * @n 0x06表示用24字节三DES X9.19算
        MAC
        * @n 0x07表示用24字节三DES POS ECB算
        MAC
        * @see #dc_CreatDesData(int, int,
        String) {@link #dc_CreatSm4Data(int,
        int, String)}
        */
public static native String
dc_CreatMacDataECB(int keyType, int
enMode, String sourceDataHex);

/**
        * @return 成功: "0000|返回的序列号字符
        串" <br>

```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 获取设备序列号。
```

```
    * @par 说明: 获取设备内部定制的序列号,  
设备默认序列号为空, 只有预先定制的设备才会存在  
可用的序列号。
```

```
    */
```

```
public static native String  
dc_readdevsnr();
```

```
/**
```

```
    * @param line    行号。
```

```
    * @param offset 偏移。
```

```
    * @param data    要显示的字符串。
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 液晶显示。
```

```
    * @par 说明: 显示指定字符串到液晶屏幕  
上。
```

```
    */
```

```
public static native String  
dc_dispinfo(int line, int offset,  
String data);
```

```
/**
 * @param line 行号。
 * @param offset 偏移。
 * @param data 要显示的字符串（中文请
传入GBK编码，如：byte[] data = "汉
字".getBytes("GBK"））。
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 液晶显示(支持中文)。
 * @par 说明：显示指定字符串到液晶屏幕
上。
 */
public static native String
dc_dispinfoSrc(int line, int offset,
byte[] data);
```

```
/**
 * @param line 需要清除的行号，编号从1
开始。
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
```

```

        * @brief LCD清屏。
        * @par 说明：清除LCD屏幕的显示内容。
        */
public static native String
dc_lcdclrscrn(int line);

/**
    * @param bOpenFlag true-开背光，
false-关背光。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief LCD背光控制。
    * @par 说明：
    * 控制设备的LCD背光。
    */
public static native String
dc_ctlbacklight(boolean bOpenFlag);

/**
    * @param offset 偏移地址
    * @param length 读取长度
    * @return 成功: "0000|读取到的数据
(HEX) " <br>

```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 读EEPROM。
```

```
    * @par 读取设备内部EEPROM中的数据，可  
    以用作数据保存等
```

```
    */
```

```
public static native String  
dc_srd_eeprom_hex(int offset, int  
length);
```

```
/**
```

```
    * @param offset 偏移地址
```

```
    * @param dataHex 写入的数据 HEX字符串  
    格式
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 写EEPROM。
```

```
    * @par 写入数据到设备内部EEPROM中，可  
    以用作数据保存等。
```

```
    */
```

```
public static native String  
dc_swr_eeprom_hex(int offset, String  
dataHex);
```



```
/**
    * @param apduHex 发送的APDU数据
    * @param timeout 超时值，此值只在部分
    设备的底层使用，单位为250毫秒，一般调用建议值
    为7。
    * @return 成功: "0000|响应APDU ( HEX
    )" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.deccard.driver.references.ErrorCode}
    <br>
    * @brief 非接触式CPU卡指令交互。
    * @par 说明：对感应区CPU卡进行指令交互
    操作，注意此接口不封装卡协议部分。
    */
public static native String
dc_pro_commandsource_int(String
apduHex, int timeout);

/**
    * @return 成
    功: "0000|UID2|UID3|UID4|UID5|..." <br>
    * 失败: "错误码|错误提示信息" {@link
    com.deccard.driver.references.ErrorCode}
    <br>
    * @brief 获取多张Type A卡UID。
```

```

        * @par 说明：获取多张Type A卡UID。
    */
public static native String
dc_RequestMultiCard();

/**
    * @param uidHex 传入UID。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 选择一张Type A卡并激活。
    * @par 说明：选择一张Type A卡并激活。
    */
public static native String
dc_ActivateCard(String uidHex);

/**
    * @param cardType 0 : typeA , 1 :
typeB
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 配置非接触卡类型

```

* @par 说明：选配置需要操作什么类型的非接触式卡，设备上电后默认操作Type A卡，可以使用此接口来改变类型，一般在寻卡前调用此接口

*/

```
public static native String  
dc_config_card(int cardType);
```

/**

* @param type 类型，0x00表示停止播放，0x01表示播放内容为UCS2编码，0x02表示播放内容为GBK编码。

* @param content 播放内容，参数是0x00时，播放内容传入 new byte[1]。

* @return 成功: "0000|"

* 失败: "错误码|错误提示信息" {@link com.deccard.driver.references.ErrorCode}

* @brief TTS语音播放。

* @par 说明： TTS语音播放。

*/

```
public static native String  
dc_TtsVoicePlay(int type, byte[]  
content);
```

/**

```
    * @param type 类型，0x01表示设置语
速，0x02表示设置音量，0x03表示设置音色。
    * @param value 增益值（0x00~0xFF），
正常为0x80。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief TTS语音设置。
    * @par 说明: TTS语音设置。
    */
```

```
public static native String
dc_TtsVoiceConfig(int type, int value);
```

```
/**
    * @param type 0 : 表示扫码一次有效。
    * @param value type = 0 时 此参数无
效，传入为0即可。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 配置二维码工作模式。
    * @par 说明: 配置二维码工作模式。
    * @n 1 : 表示连续扫码。
    * @n 2 : 表示照明设置。
```

```

    * @n type = 1 时 此参数有效，表示连续
    扫码时间间隔（单位/毫秒）。

    * @n type = 2 时 此参数有效，value =
    0，表示关闭照明，value = 1 时，表示开启照明。

    * @deprecated 此接口因固件升级不推荐使用

    */
public static native String
dc_Config2DBarcode(int type, int
value);

/**
    * @param time_ms 设备超时值，单位为毫
    秒。

    * @return 成功: "0000|二维码数据"
    <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decord.driver.references.ErrorCode}
    <br>
    * @brief 扫描二维码。
    * @par 说明: 扫描二维码。
    * @deprecated 此接口因固件升级不推荐使
    用

    */
public static native String
dc_Scan2DBarcode(int time_ms);
```

```
/**
 * @param mode 保留，固定为0x00。
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 开始扫描二维码。
 * @par 说明: 开始扫描二维码。
 */
```

```
public static native String
dc_Scan2DBarcodeStart(int mode);
```

```
/**
 * @return 成功: "0000|二维码数据"
<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取二维码数据。
 * @par 说明: 获取二维码数据。
 */
```

```
public static native String
dc_Scan2DBarcodeGetData();
```

```
/**
```

```
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 退出扫描二维码。
    * @par 说明: 退出扫描二维码。。
    */
```

```
public static native String
dc_Scan2DBarcodeExit();
```

```
/**
    * @param mCallback 监听器 {@link
com.decard.driver.references.DCCallback
.BarcodeCallback}
    * @brief 设置二维码扫描结果监听器。
    * @par 说明: 设置监听器, 接收二维码扫描
结果。
    * @deprecated 此接口因固件升级不推荐使用
    */
```

```
public static void
dc_SetBarcodeCallback(DCCallback.Barcode
Callback mCallback);
```

```

private static
DCCallback.BarcodeCallback
mBaBarcodeCallback;

/**
 * @param msgSrc 由jni层返回的结果。
 * @brief 二维码扫描结果调用函数。
 * @par 说明：此方法由JNI层返回结果调用，应用层不做调用。
 * @attention 此方法由JNI层返回结果调用，应用层不做调用。
 * @deprecated 此接口因固件升级不推荐使用
 */
public static void
dc_BardCodeReceive(byte[] msgSrc) {
    if (mBaBarcodeCallback != null) {
        String msg = new
String(msgSrc);

        mBaBarcodeCallback.Receive(msg);
    }
}

/**

```



```

        * @return 成功: "0000|位置状态"
<br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 检测卡位置状态。
        * @par 说明:
        * 检测卡片当前的位置状态。
        * @n 00 - 无卡。
        * @n 01 - 无卡, 卡在前门口, 处夹卡位
置。
        * @n 02 - 无卡, 卡在前门口, 处不夹卡位
置。
        * @n 10 - 有卡, 不可操作任何卡。
        * @n 11 - 有卡, 可操作磁条。
        * @n 12 - 有卡, 可操作接触。
        * @n 14 - 有卡, 可操作非接触。
        */
public static native String
dc_SelfServiceDeviceCardStatus();

/**
        * @return 成功: "0000|结果" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 进入卡片。

```

```

* @par 说明:
* 进入卡片, 操作前设备内有卡则错误。
* @param[in] time_s 设备超时值, 单位为秒。

* @param[in] mode 模式。
* @n 0 - 前端进卡, 不带磁条。
* @n 1 - 前端进卡, 带磁条。
* @n 2 - 后端进卡, 不带磁条。
* @n 3 - 后端进卡, 带磁条。
* @n 0001 - 设备内已有卡。
* @n 0002 - 接收超时。
* @n 0003 - 读磁卡错误。
* @n 0004 - 参数设置出错。
* @n 0005 - 异常卡正常弹出。
* @n 0006 - 异常卡卡在设备内。
*/

public static native String
dc_SelfServiceDeviceCardInject(int
time_s, int mode);

/**
* @return 成功: "0000|结果" <br>
* 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}
<br>
* @brief 弹出卡片。
* @par 说明:

```

```
* 弹出卡片，操作前设备内无卡则错误。
* @param[in] time_s 设备超时值，单位为秒。
* @param[in] mode 模式。
* @n 0 - 弹到前端并夹卡。
* @n 1 - 弹道后端。
* @n 2 - 弹到前端不夹卡。
* @n 0001 - 设备内没有卡。
* @n 0002 - 接收超时。
* @n 0003 - 参数设置出错。
* @n 0004 - 卡片已到前门夹卡位置，可操作卡片。
* @n 0005 - 卡片已到前门不夹卡位置，不可操作卡片。
*/
public static native String
dc_SelfServiceDeviceCardEject(int
time_s, int mode);

/**
* @return 成功: "0000|结果" <br>
* 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
* @brief 移动卡片。
* @par 说明:
* 移动卡片，操作前设备内无卡则错误。
```

```

    * @param[in] time_s 设备超时值，单位为秒。
    * @param[in] mode 模式。
    * @n 0 - 移动到磁条卡操作位置。
    * @n 1 - 移动到接触卡操作位置。
    * @n 2 - 移动到非接触卡操作位置。
    * @n 0001 - 设备内没有卡。
    * @n 0002 - 接收超时。
    * @n 0003 - 参数设置出错。
    * @n 0004 - 卡片已到前门不夹卡位置，不可操作卡片。
    */
public static native String
dc_SelfServiceDeviceCardMove(int
time_s, int mode);

/**
    * @return 成功: "0000|结果" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 获取传感器状态。
    * @par 说明:
    * 获取设备传感器的状态。
    * @n 0001 表示掉电无法使用。
    * @n bit0 - 电闸门开关传感器，0表示打开，1表示关闭。

```

```

        * @n bit1 - 压卡传感器，0表示卡已被下
        压，1表示无卡。

        * @n bit2~bit7 - 分别表示从前端到后端的
        传感器，每个传感器值0表示有卡，值1表示无卡。

        */
public static native String
dc_SelfServiceDeviceSensorStatus();

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    <br>

    * @brief 设置弹卡模式。
    * @par 说明:
    * 设置弹卡模式。
    * @param[in] mode 模式。
    * @n 0 - 弹到前端并夹卡。
    * @n 1 - 弹道后端。
    * @n 2 - 弹到前端不夹卡。
    */
public static native String
dc_SelfServiceDeviceConfig(int mode);

/**
    * @return 成功: "0000|" <br>

```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 设置前端进卡模式。
```

```
    * @par 说明:
```

```
    * 设置前端进卡模式。
```

```
    * @param[in] mode 模式。
```

```
    * @n 0 - 前端禁止进卡。
```

```
    * @n 1 - 前端磁卡方式进卡。
```

```
    * @n 2 - 前端开关方式进卡。
```

```
    * @n 3 - 前端磁信号方式进卡。
```

```
    */
```

```
public static native String  
dc_SelfServiceDeviceConfigFront(int  
mode);
```

```
/**
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 设置后端进卡模式。
```

```
    * @par 说明:
```

```
    * 设置后端进卡模式。
```

```
    * @param[in] mode 模式。
```

```
    * @n 0 - 后端禁止进卡。
```

```
    * @n 1 - 后端磁卡方式进卡。
```

```
        * @n 2 - 后端非磁卡方式进卡。  
        */  
public static native String  
dc_SelfServiceDeviceConfigBack(int  
mode);  
  
/**  
    * @return 成功: "0000|" <br>  
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>  
    * @brief 设置停卡位置。  
    * @par 说明:  
    * 设置停卡位置。  
    * @param[in] mode 模式。  
    * @n 0 - 前端不夹卡。  
    * @n 1 - 前端夹卡。  
    * @n 2 - 接触式IC卡位置。  
    * @n 3 - 射频卡位置。  
    * @n 4 - 磁卡位置。  
    * @n 5 - 弹卡到后端。  
    */  
public static native String  
dc_SelfServiceDeviceConfigPlace(int  
mode);  
  
/**
```

```
    * @return 成功: "0000|结果"    <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
```

```
    * @brief 检测卡类型。
```

```
    * @par 说明:
```

```
    * 设备将自动移动卡片并且检测卡片类型。
```

```
    * @n 0000 - 表示无法检测到相应卡片。
```

```
    * @n 0001 - 表示设备内无卡。
```

```
    * @n 0011 - 表示Type A CPU Card。
```

```
    * @n 0013 - 表示Type A Mifare S50。
```

```
    * @n 0014 - 表示Type A Mifare S70。
```

```
    * @n 0015 - 表示Type A Mifare
Ultralight。
```

```
    * @n 0021 - 表示Type B CPU Card。
```

```
    * @n 0022 - 表示Type B 存储卡。
```

```
    * @n 0031 - 表示接触T=0 CPU Card。
```

```
    * @n 0032 - 表示接触T=1 CPU Card。
```

```
    * @n 0041 - 表示4442 Card。
```

```
    * @n 0042 - 表示4428 Card。
```

```
    */
```

```
public static native String
dc_SelfServiceDeviceCheckCardType();
```

```
/**
```

```
    * @return 成功: "0000|结果"    <br>
```



```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 复位自助设备。
```

```
    * @par 说明:
```

```
    * 使自助设备进入上电初始状态, 并且设置参  
数为缺省参数。
```

```
    */
```

```
public static native String  
dc_SelfServiceDeviceReset();
```

```
/**
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 设置打印机参数。
```

```
    * @par 说明:
```

```
    * 设置操作打印机的参数。
```

```
    * @param[in] fontSize 字体设置, 0x00  
表示字符大小为8*16, 0x01表示字符大小为  
12*24, 0x02表示字符大小为16*32。
```

```
    * @param[in] alignment 对齐设置,  
0x00表示左对齐, 0x01表示居中, 0x02表示右对  
齐。
```

* @param[in] leftMargin 左边距设置，字符大小为8*16时<48，字符大小为12*24时<32，字符大小为16*32时<24。

* @param[in] rightMargin 右边距设置，字符大小为8*16时<48，字符大小为12*24时<32，字符大小为16*32时<24。

* @param[in] rowPitch 行间隔设置，单位为点。

* @param[in] printOutRate 打印速度设置（0x00~0x03），0x00最快，0x03最慢。

*/

```
public static native String  
dc_setprint(int fontSize, int  
alignment, int leftMargin, int  
rightMargin, int rowPitch, int  
printOutRate);
```

/**

* @return 成功: "0000|字体设置|对齐设置|左边距设置|行间隔设置|打印速度设置" 参数说明参考dc_setprint方法 {@link

#dc_setprint}

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 查询打印机参数。

```

    * @par 说明:
    * 查询操作打印机的参数。
    */
public static native String
dc_checkprint();

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 打印字符。
    * @par 说明:
    * 打印文本字符信息。
    * @param[in] character 字符数据(编码
格式“gbk”, 字符数据的长度, 必须小于或等于
480)。
    */
public static native String
dc_printcharacter(byte[] character);

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 打印一维码。

```

- * @par 说明:
- * 打印一维码信息。
- * @param[in] height 一维码的高度。
- * @param[in] displayFlag 显示类型，0x00表示图形在下文本在上，0x01表示图形在上文本在下，0x02表示只显示图形不显示文本。
- * @param[in] fontSize 字体设置，0x00表示字符大小为8*16，0x01表示字符大小为12*24，0x02表示字符大小为16*32。
- * @param[in] onedimensional 文本数据（编码格式“gbk”，文本数据的长度，必须小于或等于15）。

```
*/  
public static native String  
dc_printOnedimensional(int height, int  
displayFlag, int fontSize, byte[]  
onedimensional);
```

```
/**  
 * @return 成功: "0000|" <br>  
 * 失败: "错误码|错误提示信息" {@link  
com.decord.driver.references.ErrorCode}  
<br>  
 * @brief 打印图片。  
 * @par 说明:  
 * 打印图片信息。
```

```

    * @param[in] length 长度，单位为字节，必须小于或等于48，不能为0。
    * @param[in] height 高度，单位为点，必须小于100，不能为0。
    * @param[in] alignment 对齐设置，0x00表示左对齐，0x01表示居中，0x02表示右对齐。
    * @param[in] leftMargin 左边距设置，必须小于或等于48。
    * @param[in] rightMargin 右边距设置，必须小于或等于48。
    * @param[in] picture 图片数据。
    */
public static native String
dc_printPicture(int length, int height,
int alignment, int leftMargin, int
rightMargin, byte[] picture);

/**
    * @param RowPitch 进纸行数，单位为点。
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decadd.driver.references.ErrorCode}
<br>
    * @brief 进纸。
    * @par 说明:
```

```

        * 进纸到设备内用于打印。
        */
public static native String
dc_printenter(int RowPitch);

/**
    * @return 成功: "0000|纸状态|温度状态|字库状态" <br>
    * 失败: "错误码|错误提示信息" {@link com.decadd.driver.references.ErrorCode}
    <br>
    * @brief 获取打印机状态。
    * @par 说明:
    * 获取打印机的状态。
    * @param[in] icdev 设备标识符。
    * @param[out] paperstatus 纸状态,
    0x00表示不缺纸, 0x01表示缺纸。
    * @param[out] tempturestatus 温度状态,
    0x00表示温度正常, 0x01表示温度过高。
    * @param[out] zkstatus 字库状态。
    * @n 0x00 - 三种字库都存在。
    * @n 0x01 - 三种字库都不存在。
    * @n 0x02 - 存在8*16字库。
    * @n 0x03 - 存在12*24字库。
    * @n 0x04 - 存在16*32字库。
    * @n 0x05 - 存在8*16字库和12*24字库。
    * @n 0x06 - 存在8*16字库和16*32字库。

```

```

        * @n 0x07 - 存在12*24字库和16*32字
库。

    */
public static native String
dc_printstatus();

/**
    * @param timeout      设备超时值，单位
    为秒。
    * @param requestHex 发送数据。(请应用
    层检查输入的数据是否是合法16进制字符串)
    * @return 成功: "0000|响应数据 ( HEX
    )" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decadd.driver.references.ErrorCode}
    <br>
    * @brief 指纹模块通讯。
    * @par 说明:
    * 与指纹模块进行通讯交互。
    */
public static native String
dc_fingerprint_io_control_hex(int
timeout, final String requestHex);

/**
    * @return 成功: "0000|" <br>

```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 数码管显示。
```

```
    * @par 说明:
```

```
    * @param[in] number 数码管编号  
(1~12)。
```

```
    * @param[in] mode 模式, 0x00表示数据  
正常显示, 0x01表示数据闪烁显示, 0x02表示数据  
不闪烁显示, 0x03表示数据带小数点显示。
```

```
    * @param[in] value 显示数据值。
```

```
    */
```

```
public static native String  
dc_LedDisplay(int number, int mode, int  
value);
```

```
/**
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 数码管显示设置。
```

```
    * @par 说明:
```

```
    * 数码管显示设置。
```

```
    * @param[in] mode 模式。
```

```
    * @n 0x00 - 左移。
```

```
    * @n 0x01 - 右移。
```



```
* @n 0x02 - 左循环。  
* @n 0x03 - 右循环。  
* @n 0x04 - 显示清除。  
* @n 0x05 - 显示置位。  
* @n 0x06 - 显示亮度增加。  
* @n 0x07 - 显示亮度降低。  
* @n 0x08 - 开显示。  
* @n 0x09 - 关显示。  
*/
```

```
public static native String  
dc_LedControl(int mode);
```

```
/**  
    * @return 成功: "0000|响应数据 ( HEX  
    )" <br>  
    * 失败: "错误码|错误提示信息" {@link  
    com.decord.driver.references.ErrorCode}  
    * @brief 获取键盘输入。  
    * @par 说明:  
    * 获取键盘按键, 同时设置输入时屏幕显示的信息。  
    * @param[in] disptype 显示类型, 此类型与设备内置显示条目相关。  
    * @param[in] line 行号, 信息将会显示在此行号指定的行中。  
    * @param[in] ctime 设备超时值, 单位为秒。
```

```
*/  
public static native String  
dc_getinputkey(int disptype, int line,  
int ctime);  
  
/**  
    * @return 成功: "0000|" <br>  
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>  
    * @brief LCD显示文字。  
    * @par 说明:  
    * 通过指定字模点阵数据, 在设备LCD显示屏  
    上显示文字。  
    * @param[in] context 上下文  
    * @param[in] line 行号。  
    * @param[in] offset 偏移。  
    * @param[in] mode 模式, 0表示正色, 1  
    表示反色。  
    * @param[in] charset 字符集。(目前只  
    能填5)  
    * @n 0 - ASCII。  
    * @n 1 - GB2312。  
    * @n 2 - GBK。  
    * @n 3 - GB18030。  
    * @n 4 - ANSI。
```

```

        * @n 5 - UNICODE (Little-Endian) 。
        * @n 6 - UNICODE (Big-Endian) 。
        * @param[in] text 文本字符串。
    */

public static native String
dc_LcdDisplayText(Context context, int
line, int offset, int mode, int
charset, String text);

/**
    * @return 成功: "0000|type 类型
('A'表示Type A卡, 'B'表示Type B卡)|rbuf 激
活信息" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 寻Type A或Type B卡并激活。
    * @par 说明:
    * 对Type A或Type B卡进行寻卡和激活。
    */

public static native String
dc_cardAB();

/**
    * @return 成功: "0000|type 卡类型,
0x0A表示Type A卡, 0x0B表示Type B卡" <br>

```

```

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 检测多卡状态。
        * @par 说明:
        * 判断是否为多张卡, 是否为Type A或Type
B卡。

        */
public static native String
dc_MulticardStatus();

/**
        * @return 成功: "0000|" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

        * @brief 设置接触式CPU卡电压。
        * @par 说明:
        * 设置接触式CPU卡电压。
        * @param[in] voltage 电压值, 0x00表
示5V, 0x01表示3.3V, 0x02表示1.8V。

        */
public static native String
dc_SetCpuVoltage(int voltage);

/**
        * @return 成功: "0000|数据" <br>

```

```

        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 密码键盘模块通讯。
        * @par 说明:
        * 与密码键盘模块进行通讯交互。
        * @param[in] sendData 发送数据。
        * @param[in] timeout 设备超时值, 单位
为100毫秒。
        */
public static native String
dc_PinpadCommunication(String sendData,
int timeout);

/**
        * @return 成功: "0000|数据" <br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 读取社保卡05文件
        * @par 说明: 支持三代卡
        */
public static native String
iReadSIEF05();

/**
        * @return 成功: "0000|数据" <br>

```

```
        * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>  
        * @brief 读取社保卡06文件  
        * @par 说明: 支持三代卡  
        */  
public static native String  
iReadSIEF06();  
  
/**  
        * @return 成功: "0000|返回的UID"  
<br>  
        * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>  
        * @brief 获取身份证UID。  
        * @par 说明:  
        * 获取身份证的唯一标识。  
        */  
public static native String  
dc_get_idsnr();  
  
/**  
        * @return 成功: "0000|" <br>  
        * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```

    * @brief 播放语音。
    * @par 说明：
    * 播放设备内置语音。
    * @param[in] mode 语音段号（1~12）。
    */
public static native String
dc_play_voice(int mode);

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decord.driver.references.ErrorCode}
<br>
    * @brief 播放语音。
    * @par 说明：
    * 播放设备内置语音。
    * @param[in] mode 语音段号（1~12）。
    */
public static native String
dc_playvoice(int mode);

public static native String
dc_writedevsnr(String snr);

public static native String
dc_GetDeviceUid();

```

```
/**
    * @return 成功: "0000|卡序列号"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 寻卡请求、防卡冲突、选卡操作。
    * @par 说明:
    * 内部包含了 ::dc_request
::dc_anticol11 ::dc_select
::dc_anticol12 ::dc_select2 的功能, 此接
口有设计缺陷, 不能返回卡序列号实际长度, 建议使
用 ::dc_card_n 替代。
    * @param[in] _Mode 模式, 同
::dc_request 的 @a _Mode 。
    */
public static native String
dc_card_double_hex(int _Mode);

/**
    * @return 成功: "0000|返回的数据"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 非接触式CPU卡指令交互。
    * @par 说明:
```


* 对感应区CPU卡进行指令交互操作，注意此接口已封装卡协议部分。

* @param[in] slen 发送数据的长度。

* @param[in] sendbuffer 发送数据。

* @param[in] timeout 超时值，此值只在部分设备的底层使用，单位为250毫秒，一般调用建议值为7。

* @param[in] FG 分割值，此值只在部分设备的底层使用，单位为字节，一般调用建议值为64。

*/

```
public static native String  
dc_pro_commandlinkEXT_hex(String  
sendbuffer, int timeout, int FG);
```

/**

* @return 成功: "0000|英文姓名|性别|永久居住证号码|国籍或所在地区代码|中文姓名|证件签发日期|证件终止日期|出生日期|证件版本号|当次申请受理机关代码|证件类型标识|预留项"

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 解析文字信息。

* @par 说明:

* 解析外国人永久居住证文字信息，获取相应的条目。

* @param[in] info 文字信息。

```
*/  
public static native String  
dc_ParseTextInfoForForeignerString(String info);
```

```
/**
```

```
    * @return 成功: "0000|姓名|性别|预留  
    项|出生日期|住址|公民身份号码|签发机关|证件签  
    发日期|证件终止日期|通行证号码|签发次数|预留  
    项|证件类型标识|预留项"    <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
    com.decard.driver.references.ErrorCode}  
    <br>
```

```
    * @brief 解析文字信息。
```

```
    * @par 说明:
```

```
    * 解析港澳台居民居住证文字信息，获取相应的  
    条目。
```

```
    * @param[in] info 文字信息。
```

```
*/
```

```
public static native String  
dc_ParseTextInfoForHkMoTwString(String  
info);
```

```
/**
```

```
    * @return 成功: "0000|姓名|性别|民族|  
    出生日期|住址|公民身份号码|签发机关|证件签发日  
    期|证件终止日期|预留项"    <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 解析文字信息。
```

```
    * @par 说明:
```

```
    * 解析中国居民身份证文字信息, 获取相应的  
    条目。
```

```
    * @param[in] info 文字信息。
```

```
    */
```

```
public static native String  
dc_ParseTextInfoString(String info);
```

```
    /**
```

```
    * @brief 解析文字信息。
```

```
    * @par 说明:
```

```
    * 解析外国人永久居留证(新版)文字信息,  
    获取相应的条目。
```

```
    * @param[in] icdev 设备标识符。
```

```
    * @param[in] charset 获取条目将采用的  
    字符集, 0表示GBK, 1表示UCS-2LE, 2表示UTF-  
    8。
```

```
    * @param[in] info_len 文字信息的长  
    度。
```

```
    * @param[in] info 文字信息。
```

```
    * @param[out] chinese_name 中文姓  
    名, 请至少分配64个字节。
```

* @param[out] sex 性别代码，请至少分配8个字节。

* @param[out] renew_count 换证次数，请至少分配12个字节。

* @param[out] birth_day 出生日期，请至少分配36个字节。

* @param[out] english_name 英文姓名，请至少分配144个字节。

* @param[out] id_number 证件号码，请至少分配76个字节。

* @param[out] reserved 预留项，请至少分配64个字节。

* @param[out] expire_start_day 有效期起始日期，请至少分配36个字节。

* @param[out] expire_end_day 有效期截止日期，请至少分配36个字节。

* @param[out] english_name_ex 英文姓名备用，请至少分配48个字节。

* @param[out] citizenship 国籍代码，请至少分配16个字节。

* @param[out] type_sign 证件类型标识，请至少分配8个字节。

* @param[out] prev_related_info 既往版本外国人永久居留证件号码关联项，请至少分配16个字节。

* @param[out] old_id_number 旧永久居留证号码，请至少分配64个字节。

```

        * @return <0表示失败, ==0表示成功。
    */
    public static native short
dc_ParseTextInfoForNewForeigner(int
charset,int info_len,byte[] info,byte[]
chinese_name,byte[] sex,byte[]
renew_count,byte[] birth_day,byte[]
english_name,byte[] id_number,byte[]
reserved,byte[]
expire_start_day,byte[] expire_end_day,byte[]
english_name_ex,byte[]
citizenship,byte[] type_sign,byte[]
prev_related_info,byte[]
old_id_number);

```

```

/**
    * @return <0表示失败, ==0表示成功。
    * @brief 解析文字信息。
    * @par 说明:
    * 解析中国居民身份证文字信息, 获取相应的
    的条目。
    * @param[in] charset 获取条目将采用的
    字符集, 0表示GBK, 1表示UCS-2LE, 2表示UTF-
    8。
    * @param[in] info_len 文字信息的长
    度。
    * @param[in] info 文字信息。

```

* @param[out] name 姓名，请至少分配64个字节。

* @param[out] sex 性别，请至少分配8个字节。

* @param[out] nation 民族，请至少分配12个字节。

* @param[out] birth_day 出生日期，请至少分配36个字节。

* @param[out] address 住址，请至少分配144个字节。

* @param[out] id_number 公民身份号码，请至少分配76个字节。

* @param[out] department 签发机关，请至少分配64个字节。

* @param[out] expire_start_day 证件签发日期，请至少分配36个字节。

* @param[out] expire_end_day 证件终止日期，请至少分配36个字节。

* @param[out] reserved 预留项，请至少分配76个字节。

*/

```
public static native short  
dc_ParseTextInfo(int charset, int  
info_len, byte[] info, byte[] name,  
byte[] sex, byte[] nation, byte[]  
birth_day, byte[] address, byte[]  
id_number, byte[] department, byte[]  
expire_start_day, byte[]  
expire_end_day, byte[] reserved);
```

```
/**
```

```
    * @return <0表示失败，==0表示成功。
```

```
    * @brief 解析文字信息。
```

```
    * @par 说明：
```

```
    * 解析外国人永久居留证文字信息，获取相应  
    的条目。
```

```
    * @param[in] charset 获取条目将采用的  
    字符集，0表示GBK，1表示UCS-2LE，2表示UTF-  
    8。
```

```
    * @param[in] info_len 文字信息的长  
    度。
```

```
    * @param[in] info 文字信息。
```

```
    * @param[out] english_name 英文姓  
    名，请至少分配244个字节。
```

```
    * @param[out] sex 性别，请至少分配8个  
    字节。
```

```
    * @param[out] id_number 永久居留证号  
    码，请至少分配64个字节。
```

* @param[out] citizenship 国籍或所在地区代码，请至少分配16个字节。

* @param[out] chinese_name 中文姓名，请至少分配64个字节。

* @param[out] expire_start_day 证件签发日期，请至少分配36个字节。

* @param[out] expire_end_day 证件终止日期，请至少分配36个字节。

* @param[out] birth_day 出生日期，请至少分配36个字节。

* @param[out] version_number 证件版本号，请至少分配12个字节。

* @param[out] department_code 当次申请受理机关代码，请至少分配20个字节。

* @param[out] type_sign 证件类型标识，请至少分配8个字节。

* @param[out] reserved 预留项，请至少分配16个字节。

*/


```
public static native short
dc_ParseTextInfoForForeigner(int
charset, int info_len, byte[] info,
byte[] english_name, byte[] sex, byte[]
id_number, byte[] citizenship, byte[]
chinese_name, byte[] expire_start_day,
byte[] expire_end_day, byte[]
birth_day, byte[] version_number,
byte[] department_code, byte[]
type_sign, byte[] reserved);
```

```
/**
```

- * @return <0表示失败，==0表示成功。

- * @brief 解析文字信息。

- * @par 说明：

- * 解析港澳台居民居住证文字信息，获取相应的条目。

- * @param[in] icdev 设备标识符。

- * @param[in] charset 获取条目将采用的字符集，0表示GBK，1表示UCS-2LE，2表示UTF-8。

- * @param[in] info_len 文字信息的长度。

- * @param[in] info 文字信息。

- * @param[out] name 姓名，请至少分配64个字节。

- * @param[out] sex 性别，请至少分配8个字节。
- * @param[out] reserved 预留项，请至少分配12个字节。
- * @param[out] birth_day 出生日期，请至少分配36个字节。
- * @param[out] address 住址，请至少分配144个字节。
- * @param[out] id_number 公民身份号码，请至少分配76个字节。
- * @param[out] department 签发机关，请至少分配64个字节。
- * @param[out] expire_start_day 证件签发日期，请至少分配36个字节。
- * @param[out] expire_end_day 证件终止日期，请至少分配36个字节。
- * @param[out] pass_number 通行证号码，请至少分配40个字节。
- * @param[out] sign_count 签发次数，请至少分配12个字节。
- * @param[out] reserved2 预留项，请至少分配16个字节。
- * @param[out] type_sign 证件类型标识，请至少分配8个字节。
- * @param[out] reserved3 预留项，请至少分配16个字节。
- */

```
public static native short
dc_ParseTextInfoForHkMoTw(int charset,
int info_len, byte[] info, byte[] name,
byte[] sex, byte[] reserved, byte[]
birth_day, byte[] address, byte[]
id_number, byte[] department, byte[]
expire_start_day, byte[]
expire_end_day, byte[] pass_number,
byte[] sign_count, byte[] reserved2,
byte[] type_sign, byte[] reserved3);
```

```
/**
```

- * @return <0表示失败，==0表示成功。

- * @brief 解析相片信息。

- * @par 说明：

- * 解析相片信息，通过公安部相片解码库还原相片图像数据。

- * @param[in] icdev 设备标识符。

- * @param[in] type 相片图像数据的格式，0表示BMP文件，1表示BMP缓存，2表示BMP Base64字符串。

- * @param[in] info_len 相片信息的长度。

- * @param[in] info 相片信息。

- * @param[in,out] photo_len 数据的长度。

- * @n BMP文件 - 无效。

- * @n BMP缓存 - 传入 @a photo 分配的字节数，返回相片图像数据的长度。
- * @n BMP Base64字符串 - 传入 @a photo 分配的字节数，返回相片图像数据Base64字符串的长度，不含'\0'。
- * @param[in,out] photo 数据。
- * @n BMP文件 - 传入文件名，请确保有写入的权限。
- * @n BMP缓存 - 返回的相片图像数据，请至少分配65536个字节。
- * @n BMP Base64字符串 - 返回的相片图像数据Base64字符串，请至少分配65536个字节。

```

*/
public static native short
dc_ParsePhotoInfo(int type, int
info_len, byte[] info, byte[]
photo_len, byte[] photo);

/**
* @return <0表示失败，==0表示成功。
* @brief 解析其它信息。
* @par 说明：
* 解析其它信息。
* @param[in] icdev 设备标识符。
* @param[in] flag 标志。
* @n 当信息采用GBK编码时，使用0x00表示
性别，0x01表示民族，0x02表示国籍。

```

```

        * @n 当信息采用UCS-2LE编码时，使用0x10
表示性别，0x11表示民族，0x12表示国籍。

        * @n 当信息采用UTF-8编码时，使用0x20表
示性别，0x21表示民族，0x22表示国籍。

        * @param[in] in_info 传入信息。
        * @param[out] out_info 返回的信息。
    */
public static native short
dc_ParseOtherInfo(int flag, byte[]
in_info, byte[] out_info);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 外接密码键盘通讯。
    * @par 说明:
    * 与外置密码键盘进行通讯交互。
    * @param[in] ctime 设备超时值，单位为
秒。

    * @param[in] source 发送数据。
    */
public static native String
SD_InstallKey(int ctime, String
source);

```

```
/**
 * @return 成功: "0000|返回的版本字符串" <br>
 * 失败: "错误码|错误提示信息" {@link
 com.decard.driver.references.ErrorCode}
 <br>
 * @brief 获取外接密码键盘的版本。
 * @par 说明:
 * 获取外接密码键盘的版本。
 */
public static native String
SD_IFD_GetVersion();
```

```
/**
 * @return 成功: "0000|序列号字符串"
 <br>
 * 失败: "错误码|错误提示信息" {@link
 com.decard.driver.references.ErrorCode}
 <br>
 * @brief 获取外接密码键盘设备序列号。
 * @par 说明:
 * 获取外接密码键盘设备内部定制的序列号，
 设备默认序列号为空，只有预先定制的设备才会存在
 可用的序列号。
 */
```

```

public static native String
SD_IFD_ReadSerialNumber();

/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 外接密码键盘播放语音。
 * @par 说明:
 * 播放外接密码键盘内置语音。
 * @param[in] _playmode 语音段号
(0x01~0x08) 。
 * @n 0x01 - "请输入密码"。
 * @n 0x02 - "请输入原密码"。
 * @n 0x03 - "请再输入一次"。
 * @n 0x04 - "请输入新密码"。
 * @n 0x05 - "请确认新密码"。
 * @n 0x06 - "密码修改成功"。
 * @n 0x07 - "请插卡"。
 * @n 0x08 - "请刷卡"。
 */
public static native String
SD_IFD_PlayVoice(int playMode);

/**
 * @return 成功: "0000|" <br>

```

```

    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 外接密码键盘初始化。
    * @par 说明:
    * 对外接密码键盘进行初始化操作, 使其复位
为出厂状态, 出厂状态下所有的主密钥内存空间都为
字节0x88。
    */
public static native String
SD_IFD_Init();

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 设置外接密码键盘主密钥。
    * @par 说明:
    * 对外接密码键盘内的主密钥进行设置操作。
    * @param[in] keyset 密钥组号, 从0x00
开始编号。
    * @param[in] oldkey 旧主密钥数据, 明
文, 固定为16个字节。
    * @param[in] newkey 新主密钥数据, 明
文, 固定为16个字节。
    */
```



```
public static native String
SD_IFD_SetMainKey(int keyset, String
oldkey, String newkey);

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 设置外接密码键盘工作密钥。
    * @par 说明:
    * 对外接密码键盘内的工作密钥进行设置操
作。
    * @param[in] keysetmain 主密钥组号,
从0x00开始编号。
    * @param[in] keysetwork 工作密钥组
号, 从0x00开始编号。
    * @param[in] enkeywork 工作密钥数
据, 密文 (用主密钥加密), 固定为16个字节。
    */
public static native String
SD_IFD_SetWorkKey(int keysetmain, int
keysetwork, String enkeywork);

/**
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 激活外接密码键盘工作密钥。
```

```
    * @par 说明:
```

```
    * 激活外接密码键盘当前使用的工作密钥。
```

```
    * @param[in] keysetmain 主密钥组号,  
从0x00开始编号。
```

```
    * @param[in] keysetwork 工作密钥组  
号, 从0x00开始编号。
```

```
    */
```

```
public static native String  
SD_IFD_ActWorkKey(int keysetmain, int  
keysetwork);
```

```
/**
```

```
    * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 设置外接密码键盘最大输入长度。
```

```
    * @par 说明:
```

```
    * 设置外接密码键盘允许用户输入密码的最大  
长度。
```

```
    * @param[in] keylength 密码最大长度  
(1~16)。
```

```
    */
```

```
public static native String
SD_IFD_SetKeyLength(int keylength);

/**
 * @return 成功: "0000|响应数据 ( HEX
)" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取外接密码键盘明文数据。
 * @par 说明:
 * 从外接密码键盘获取明文按键数据。
 * @param[out] szPasswd 返回的按键数
据, 数据格式为'\0'结尾的字符串。
 */
public static native String
SD_IFD_GetPIN();

/**
 * @return 成功: "0000|响应数据 ( HEX
)" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取外接密码键盘密文数据。
 * @par 说明:
 * 从外接密码键盘获取密文按键数据。
```

```
    * @param[out] szPasswd 返回的按键数据，数据为密文的HEX格式字符串。
    */
public static native String
SD_IFD_GetEnPIN();

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 获取外接密码键盘明文数据。
    * @par 说明:
    * 从外接密码键盘获取明文按键数据。
    * @param[in] yyflag 标记。
    * @n 0x80 - 绿灯亮，液晶显示"欢迎使用"，无语音提示。
    * @n 0x81 - 绿灯亮，液晶显示"请再输入一次"，有语音提示。
    * @n 0x82 - 绿灯亮，液晶显示"请输入密码"，有语音提示。
    * @n 0x83 - 绿灯亮，液晶不操作，无语音提示。
    * @n 0x84 - 绿灯亮，液晶显示"请输入原密码"，有语音提示。
```

```

    * @n 0x85 - 绿灯亮，液晶显示"请输入新
密码"，有语音提示。

    * @n 0x86 - 绿灯亮，液晶显示"请输入新
密码"，无语音提示。

    * @param[in] timeout 设备超时值，单位
为秒。

    */
public static native String
SD_IFD_GetPINPro(int yyflag, int
timeout);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 获取外接密码键盘密文数据。
    * @par 说明:
    * 从外接密码键盘获取密文按键数据。
    * @param[in] yyflag 标记。
    * @n 0x30 - 密码键盘提示"请输入密码",
语音提示，输入密码显示 '*'。
    * @n 0x31 - 密码键盘提示"请再输入一
次"，语音提示，输入密码显示 '*'。
    * @n 0x32 - 密码键盘提示"请输入密码",
无语音提示，输入密码显示 '*'。

```

```

    * @n 0x33 - 密码键盘提示"请再输入一次", 无语音提示, 输入密码显示' *'。

    * @n 0x34 - 密码键盘提示"请输入密码", 语音提示, 输入密码明文显示。

    * @n 0x35 - 密码键盘提示"请再输入一次", 语音提示, 输入密码明文显示。

    * @n 0x36 - 密码键盘提示"请输入密码", 无语音提示, 输入密码明文显示。

    * @n 0x37 - 密码键盘提示"请再输入一次", 无语音提示, 输入密码明文显示。

    * @param[in] modeflag 模式。
    * @n 0x30 - 按"确认"键或到达指定的PIN长度。

    * @n 0x31 - 按"确认"键。
    * @n 0x32 - 到达指定的PIN长度。
    * @n 0x33 - 到达指定的PIN长度并按"确认"键。

    * @param[in] timeout 设备超时值, 单位为秒。

    */
public static native String
SD_IFD_GetEnPINPro(int yyflag, int
modeflag, int timeout);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
```

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 获取外接密码键盘密文数据。

* @par 说明:

* 从外接密码键盘获取ANSI X9.8 PIN BLOCK加密的按键数据。

* @param[in] yyflag 标记。

* @n 0x30 - 密码键盘提示"请输入密码", 语音提示, 输入密码显示 '*'。

* @n 0x31 - 密码键盘提示"请再输入一次", 语音提示, 输入密码显示 '*'。

* @n 0x32 - 密码键盘提示"请输入密码", 无语音提示, 输入密码显示 '*'。

* @n 0x33 - 密码键盘提示"请再输入一次", 无语音提示, 输入密码显示 '*'。

* @n 0x34 - 密码键盘提示"请输入密码", 语音提示, 输入密码明文显示。

* @n 0x35 - 密码键盘提示"请再输入一次", 语音提示, 输入密码明文显示。

* @n 0x36 - 密码键盘提示"请输入密码", 无语音提示, 输入密码明文显示。

* @n 0x37 - 密码键盘提示"请再输入一次", 无语音提示, 输入密码明文显示。

* @param[in] modeflag 模式。

* @n 0x30 - 按"确认"键或到达指定的PIN长度。

```

        * @n 0x31 - 按"确认"键。
        * @n 0x32 - 到达指定的PIN长度。
        * @n 0x33 - 到达指定的PIN长度并按"确
        认"键。

        * @param[in] cardno 截取后的卡号，固
        定为12个字节。
        * @param[in] timeout 设备超时值，单位
        为秒。
    */
    public static native String
    SD_IFD_GetEnPINBlock(int yyflag, int
    modeflag, String cardno, int timeout);

    /**
        * @return 成功: "0000|响应数据 ( HEX
        )" <br>
        * 失败: "错误码|错误提示信息" {@link
        com.decard.driver.references.ErrorCode}
        <br>
        * @brief 设置外接密码键盘算法。
        * @par 说明:
        * 设置外接密码键盘使用的算法。
        * @param[in] desmode 模式, 0x01表示
        单DES, 0x02表示三DES。
    */
    public static native String
    SD_IFD_SelectDes(int desmode);

```



```
/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 外接密码键盘液晶显示。
    * @par 说明:
    * 显示指定字符串到外接密码键盘的液晶屏幕
上。

    * @param[in] line 行号。
    * @param[in] row 偏移。
    * @param[in] leddata 要显示的字符串。
    * @param[in] timeout 设备超时值, 单位
为秒。
*/
public static native String
SD_IFD_LEDDisplay(int line, int row,
String leddata, int timeout);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
```

```

        * @brief 用外接密码键盘工作密钥加解密。
        * @par 说明：
        * 使用外接密码键盘的工作密钥进行数据加解密操作。

        * @param[in] mode 运算模式，0x00表示加密，0x01表示解密。
        * @param[in] srcdata 传入数据，HEX格式字符串。
        */
public static native String
SD_IFD_DES(int mode, String srcdata);

/**
    * @return 成功: "0000|响应数据 ( HEX )" <br>
    * 失败: "错误码|错误提示信息" {@link com.decord.driver.references.ErrorCode}
    <br>
    * @brief 用外接密码键盘工作密钥加解密。
    * @par 说明：
    * 使用外接密码键盘的工作密钥进行数据加解密操作。

    * @param[in] srcdata 传入数据。
    * @param[in] flag 运算模式，0x00表示加密，0x01表示解密。
    */

```

```
public static native String
SD_IFD_DesCaculate(String srcdata, int
flag);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decord.driver.references.ErrorCode}
<br>
    * @brief 用外接密码键盘工作密钥算MAC。
    * @par 说明:
    * 使用外接密码键盘的工作密钥进行数据算
MAC操作。
    * @param[in] srclen 传入数据的长度。
    * @param[in] srcdata 传入数据。
    * @param[out] macdata 返回的MAC字符
串, 请至少分配64个字节。
    */
public static native String
SD_IFD_CreateMac(String srcdata);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
```

```

    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 外接密码键盘开始扫描二维码。
    * @par 说明:
    * 外接密码键盘开始扫描二维码。
    * @param[in] mode 保留, 固定为0x00。
    */
public static native String
SD_IFD_Scan2DBarcodeStart(int mode);

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 外接密码键盘获取二维码数据。
    * @par 说明:
    * 外接密码键盘获取二维码数据。
    */
public static native String
SD_IFD_Scan2DBarcodeGetData();

/**
    * @return 成功: "0000|响应数据 ( HEX
)" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 外接密码键盘退出扫描二维码。
```

```
    * @par 说明:
```

```
    * 外接密码键盘退出扫描二维码。
```

```
    */
```

```
public static native String  
SD_IFD_Scan2DBarcodeExit();
```

```
/**
```

```
    * @return 成功: "0000|响应数据 ( HEX  
)" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 外接密码键盘获取屏幕属性。
```

```
    * @par 说明:
```

```
    * 外接密码键盘获取屏幕属性。
```

```
    * @param[in] type 类别, 0x00表示获取  
设备内部字模数据缓存字节大小, 0x01表示获取屏幕  
X轴像素个数, 0x02表示获取屏幕Y轴像素个数,  
0x03表示屏幕单个像素所占用位数。
```

```
    * @n type为0x00时, 返回4字节缓存字节大  
小, 高位在前。
```

```
    * @n type为0x01时, 返回4字节屏幕X轴像  
素个数, 高位在前。
```

* @n type为0x02时，返回4字节屏幕Y轴像素个数，高位在前。

* @n type为0x03时，返回1字节屏幕单个像素所占用位数。

*/

```
public static native String  
SD_IFD_GetScreenAttribute(int type);
```

/**

* @return 成功: "0000|响应数据 (HEX)"

* 失败: "错误码|错误提示信息" {@link com.decord.driver.references.ErrorCode}

* @brief 外接密码键盘画屏。

* @par 说明:

* 外接密码键盘把像素图像画在屏幕上，只支持一次性画整个屏幕。

* @param[in] pixel 像素数据。

*/

```
public static native String  
SD_IFD_DrawScreen(String pixel);
```

```
public static native void  
setKeyboardType(int type);
```

```
public static native int
setInitParameter(int portType, String
serialPortPath, int baudRate);

public static native void
CreateAndroidContext(Context context);

public static native void
ReleaseAndroidContext();

/**
 * @return 成功: "0000|ID号字符串"
<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 获取芯片唯一ID号。
 * @par 说明:
 * 获取芯片唯一ID号。
 */
public static native String
dc_GetChipIdNumber();

/**
 * @return 成功: "0000|返回的序列号字
符串" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 读取SAM_A模块序列号。
```

```
    * @par 说明:
```

```
    * 读取SAM_A模块序列号。
```

```
    */
```

```
public static native String  
dc_SamAReadSerialNumber();
```

```
/**
```

```
    * @return 成功: "0000|返回的数据"
```

```
<br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief SAM_A模块指令交互。
```

```
    * @par 说明:
```

```
    * 此接口用于操作居民身份证验证安全控制模  
块。
```

```
    * @param[in] code 指令代码。
```

```
    * @n 0x10FF - 复位SAM_A, 无发送数据,  
无返回数据。
```

```
    * @n 0x11FF - SAM_A状态检测, 无发送数  
据, 无返回数据。
```

```
    * @n 0x12FF - 读SAM_A管理信息, 无发送  
数据, 返回数据为16个字节的SAM_A编号。
```


- * @n 0x2001 - 寻找居民身份证，无发送数据，返回数据为4个字节的0x00。
- * @n 0x2002 - 选取居民身份证，无发送数据，返回数据为8个字节的0x00。
- * @n 0x3001 - 读机读文字信息和相片信息，无发送数据，返回数据格式为：【2个字节文字信息长度（Big-Endian）】【2个字节相片信息长度（Big-Endian）】【文字信息（不大于256个字节）】【相片信息（不大于1024个字节）】。
- * @n 0x3010 - 读机读文字信息、相片信息和指纹信息，无发送数据，返回数据格式为：【2个字节文字信息长度（Big-Endian）】【2个字节相片信息长度（Big-Endian）】【2个字节指纹信息长度（Big-Endian）】【文字信息（不大于256个字节）】【相片信息（不大于1024个字节）】【指纹信息（不大于1024个字节）】。
- * @n 0x3003 - 读追加住址信息，无发送数据，返回数据为70个字节的追加住址信息。
- * @n 0x6000 - 设置UART接口速率，速率为115200bps，无发送数据，无返回数据。
- * @n 0x6001 - 设置UART接口速率，速率为57600bps，无发送数据，无返回数据。
- * @n 0x6002 - 设置UART接口速率，速率为38400bps，无发送数据，无返回数据。
- * @n 0x6003 - 设置UART接口速率，速率为19200bps，无发送数据，无返回数据。

```

        * @n 0x6004 - 设置UART接口速率，速率为9600bps，无发送数据，无返回数据。

        * @n 0x61FF - 设置SAM_A与射频模块一帧通信数据的最大字节数，发送数据为1个字节的最大字节数，取值返回为0x18~0xFF，默认值为0x58，无返回数据。

        * @param[in] sdata 发送数据。
    */
public static native String
dc_SamAControl(int code, String sdata);

/**
    * @return 成功: "0000|version 返回的版本字符串" <br>
    * 失败: "错误码|错误提示信息" {@link com.decadd.driver.references.ErrorCode} <br>
    * @brief 打开密码键盘。
    * @par 说明:
    * 打开密码键盘并且分配相关资源。。
    * @param[in] number 编号，0表示内置密码键盘，1表示外置密码键盘。
    */
public static native String
dc_KeypadOpen(int number);

/**

```

```

        * @return 成功: "0000|返回的数据"
<br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 关闭密码键盘。
        * @par 说明:
        * 关闭密码键盘并且释放相关资源。
        * @param[in] number 编号, 0表示内置密
码键盘, 1表示外置密码键盘。
        */
public static native String
dc_KeypadClose(int number);

/**
        * @return 成功: "0000|返回的数据"
<br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 装载密钥。
        * @par 说明:
        * 装载密钥到密码键盘中永久保存。
        * @param[in] number 编号, 0表示内置密
码键盘, 1表示外置密码键盘。
        * @param[in] set_index 密钥套索引
(0~15)。

```

* @param[in] sub_index 密钥索引
(0~15)，其中0表示主密钥。

* @param[in] type 密钥类型，0表示单DES，1表示三DES，2表示24字节三DES，16表示SM4。

* @param[in] mode 装载模式，0表示明文装载，1表示密文装载。

* @param[in] key_data 密钥数据。

* @param[in] flag 标志，0表示不验证，1表示验证类别1，2表示验证类别2。

* @param[in] ex_data 附加数据，含义由@a flag 的值来决定。

* @n flag为1 - ex_data为out形式的参数，返回加密全0x00后的数据。

* @n flag为2 - ex_data为In形式的参数，传入加密全0x00后的数据。

*/

```
public static native String  
dc_KeypadLoadKey(int number, int  
set_index, int sub_index, int type, int  
mode, String key_data, int flag, String  
ex_data);
```

```
/**
```

* @return 成功: "0000|返回的数据"

```
<br>
```

```

    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 清除密钥。
    * @par 说明:
    * 清除密码键盘中指定的密钥。
    * @param[in] number 编号, 0表示内置密
码键盘, 1表示外置密码键盘。
    * @param[in] set_index 密钥套索引
(0~15), -1表示全部密钥套。
    * @param[in] sub_index 密钥索引
(0~15), 其中0表示主密钥, -1表示全部密钥。
    */
public static native String
dc_KeypadClearKey(int number, int
set_index, int sub_index);

/**
    * @return 成功: "0000|密钥类型, 0表示
单DES, 1表示三DES, 2表示24字节三DES, 16表示
SM4" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

    * @brief 获取密钥类型。
    * @par 说明:
    * 获取密码键盘中指定密钥的类型。

```

```

        * @param[in] number 编号，0表示内置密码键盘，1表示外置密码键盘。

        * @param[in] set_index 密钥套索引（0~15）。

        * @param[in] sub_index 密钥索引（0~15），其中0表示主密钥。

    */
public static native String
dc_KeypadGetKeyType(int number, int
set_index, int sub_index);

/**
    * @return 成功: "0000|返回的数据"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decord.driver.references.ErrorCode}
<br>
    * @brief 数据加解密。
    * @par 说明:
    * 使用密码键盘中装载的密钥进行数据加解密操作。

    * @param[in] number 编号，0表示内置密码键盘，1表示外置密码键盘。

    * @param[in] set_index 密钥套索引（0~15）。

    * @param[in] sub_index 密钥索引（0~15），其中0表示主密钥。

```

* @param[in] flag 标志，0表示使用密钥默认算法，1表示使用密钥0~7字节算单DES，2表示使用密钥8~15字节算单DES，3表示使用密钥16~23字节算单DES。

* @param[in] mode 模式，0表示加密，1表示解密。

* @param[in] in_data 输入数据。

*/

```
public static native String  
dc_KeypadAlgorithm(int number, int  
set_index, int sub_index, int flag, int  
mode, String in_data);
```

/**

* @return 成功: "0000|返回加密数据"

* 失败: "错误码|错误提示信息" {@link
com.decaddriver.references.ErrorCode}

* @brief 获取加密按键数据。

* @par 说明:

* 从密码键盘获取ANSI X9.8 PIN BLOCK加密的按键数据。

* @param[in] icdev 设备标识符。

* @param[in] ctime 设备超时值，单位为秒。

* @param[in] pandata 卡号。

```

        * @param[in] keydata 密钥。
    */
public static native String
dc_GetEnPassAddr(int ctime, String
pandata, String keydata);

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 控制脉冲信号输出。
    * @par 说明:
    * 控制脉冲信号输出。
    * @param[in] on_off 控制开启或者关
闭, 0表示关闭, 1表示开启。
    * @param[in] occupation_ratio 占空
比, 百分比范围0~100。
    * @param[in] frequency 脉冲频率, 单
位为Hz。
    * @param[in] output_time_ms 脉冲输
出的时间, 单位为毫秒, 0表示一直输出。
    */
public static native String
dc_PulseSignalControl(int on_off, int
occupation_ratio, int frequency, int
output_time_ms);

```



```
/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 检测102卡。
 * @par 说明:
 * 检测是否存在102卡。
 */
public static native String
dc_check_102();
```

```
/**
 * @return 成功: "0000|" <br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 102卡下电。
 * @par 说明:
 * 对102卡进行下电操作。
 */
public static native String
dc_down_102();
```

```
/**
 * @return 成功: "0000|" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 写102卡。
```

```
    * @par 说明:
```

```
    * 写入数据到102卡中。
```

```
    * @param[in] offset 偏移。
```

```
    * @param[in] length 写入长度。
```

```
    * @param[in] writedata 传入数据。
```

```
    */
```

```
public static native String  
dc_write_102(int offset, int length,  
String writedata);
```

```
/**
```

```
    * @return 成功: "0000|数据" <br>
```

```
    * 失败: "错误码|错误提示信息" {@link  
com.decard.driver.references.ErrorCode}  
<br>
```

```
    * @brief 读102卡。
```

```
    * @par 说明:
```

```
    * 读取102卡的数据。
```

```
    * @param[in] offset 偏移。
```

```
    * @param[in] length 读取长度。
```

```
    * @param[out] readdata 返回的数据。
```

```
    */
```

```
public static native String
dc_read_102(int offset, int length);

/**
 * @return 成功: "0000|返回加密数据"
<br>
 * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
 * @brief 验证102卡密码。
 * @par 说明:
 * 使用传入的密码来验证102卡密码。
 * @param[in] zone 密码区。
 * @n 0 - 表示总密码, 密码长度为2个字
节。
 * @n 11 - 表示一区擦除密码, 密码长度为6
个字节。
 * @n 12 - 表示二区擦除密码, 密码长度为4
个字节。
 * @param[in] password 密码。
 */
public static native String
dc_checkpass_102(short zone, String
password);

/**
```

```

        * @return 成功: "0000|返回加密数据"
<br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
        * @brief 修改102卡密码。
        * @par 说明:
        * 修改102卡的密码。
        * @param[in] zone 密码区。
        * @n 0 - 表示总密码, 密码长度为2个字
节。
        * @n 11 - 表示一区擦除密码, 密码长度为6
个字节。
        * @n 12 - 表示二区擦除密码, 密码长度为4
个字节。
        * @param[in] password 密码。
        */
public static native String
dc_changepass_102(short zone, String
password);

/**
        * @return 成功: "0000|返回加密数据"
<br>
        * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>

```

```

    * @brief 读102卡密码计数。
    * @par 说明:
    * 读取102卡的密码计数，此计数值表示可以
    尝试验证密码的次数。
    * @param[in] zone 密码区。
    * @n 0 - 表示总密码。
    */
public static native String
dc_readcount_102(short zone);

/**
    * @return 成功: "0000|返回加密数据"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 102卡熔丝。
    * @par 说明:
    * 对102卡进行熔丝操作。
    */
public static native String
dc_fuse_102();

/**
    * @return 成功: "0000|返回加密数据"
<br>
```

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 外部模块电源控制。

* @par 说明:

* 用于对外部模块的电源进行上电或下电控制。

* @param[in] module_number 模块编号。

* @n 0x0001 - 基础外设, 如: SPI、FLASH、蜂鸣器、指示灯等。

* @n 0x0101 - Android模块。

* @param[in] value 值, 0x00表示下电, 0x01表示上电。

*/

```
public static native String  
dc_ModulePowerControl(int  
module_number, int value);
```

```
/**
```

* @return 成功: "0000|返回加密数据"

* 失败: "错误码|错误提示信息" {@link com.decard.driver.references.ErrorCode}

* @brief 外部模块心跳包控制。

* @par 说明:

* 用于开启或关闭检测外部模块心跳包功能，以便设备知晓外设模块是否仍在正常运行。

* @param[in] module_number 模块编号。

* @n 0x0001 - 基础外设，如：SPI、FLASH、蜂鸣器、指示灯等。

* @n 0x0101 - Android模块。

* @param[in] value 值，0x00表示关闭，0x01表示开启。

*/

```
public static native String  
dc_ModuleHeartbeatPacketControl(int  
module_number, int value);
```

/**

* @return 成功: "0000|返回加密数据"

* 失败: "错误码|错误提示信息" {@link
com.decord.driver.references.ErrorCode}

* @brief 通知主业务APP运行状态。

* @par 说明:

* 主业务APP用此接口来告知设备当前自己的运行状态。

* @param[in] value 值，0x00表示APP退出，0x01表示APP启动。

*/

```
public static native String
dc_NotifyMainAppState(int value);

/**
    * @return 成功: "0000|返回加密数据"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
<br>
    * @brief 防拆侦测设置。
    * @par 说明:
    * 防拆侦测设置。
    * @param[in] mode 防拆机制, 目前固定
为0x01。
    * @param[in] item 设置项目, 0x01表示
启动防拆。
    * @param[in] in_data 输入数据。
    * @param[in] in_len 输入数据的长度。
    * @param[out] out_data 输出数据。
    * @param[out] out_len 输出数据的长
度。
    */
public static native String
dc_SetTamperSense(int mode, int item,
String in_data);

/**
```


- * **@return** <0表示失败，==0表示成功。
- * **@brief** 读身份证。
- * **@par** 说明：
 - * 读取身份证的原始信息数据。
 - * **@param[in] icdev** 设备标识符。
 - * **@param[in] type** 类型。
 - * **@n 1** - 读取文字信息、相片信息和指纹信息。
 - * **@n 2** - 读取追加住址信息。
 - * **@n 3** - 读取文字信息、相片信息、指纹信息和追加住址信息。
 - * **@param[out] text_len** 返回文字信息的长度。
 - * **@param[out] text** 返回的文字信息，请至少分配256个字节。
 - * **@param[out] photo_len** 返回相片信息的长度。
 - * **@param[out] photo** 返回的相片信息，请至少分配1024个字节。
 - * **@param[out] fingerprint_len** 返回指纹信息的长度。
 - * **@param[out] fingerprint** 返回的指纹信息，请至少分配1024个字节。
 - * **@param[out] extra_len** 返回追加住址信息的长度。
 - * **@param[out] extra** 返回的追加住址信息，请至少分配70个字节。

```
*/  
public static native short  
dc_SamAReadCardInfo(int type, byte[]  
text_len, byte[] text, byte[]  
photo_len, byte[] photo, byte[]  
fingerprint_len, byte[] fingerprint,  
byte[] extra_len, byte[] extra);  
  
/**  
    * @return 成功: "0000|数据" <br>  
    * 失败: "错误码|错误提示信息" {@link  
com.deccard.driver.references.ErrorCode}  
<br>  
    * @brief 2.4G指令交互。  
    * @par 说明:  
    * 用2.4G进行指令交互操作。  
    * @param[in] icdev 设备标识符。  
    * @param[in] slen 发送数据的长度。  
    * @param[in] senddata 发送数据。  
    * @param[out] rlen 返回数据的长度。  
    * @param[out] recvbuffer 返回的数  
据。  
    * @param[in] timeout 设备超时值, 单位  
为秒。  
    */
```

```
public static native String
dc_2dot4_command(String senddata, int
timeout);

/**
    * @return 成功: "0000|" <br>
    * 失败: "错误码|错误提示信息" {@link
com.decord.driver.references.ErrorCode}
<br>
    * @brief 设置USB模式。
    * @par 说明:
    * 设置USB通讯模式，必须重新对设备进行上
电才能生效。
    * @param[in] icdev 设备标识符。
    * @param[in] ucmode 模式。
    * @n 0x00 - 表示中断发送和中断接收，此
时VID=257B，PID=3010。
    * @n 0x01 - 表示控制发送和控制接收，此
时VID=257B，PID=3011。
    * @n 0x02 - 表示控制发送和中断接收，此
时VID=0471，PID=7002。
    * @n 0x03 - 表示控制发送和中断接收，此
时VID=0471，PID=7003。
    * @n 0x04 - 表示控制发送和中断接收，此
时VID=0471，PID=7003，支持磁条卡自动上传功
能。
    * @n 0x05 - 虚拟串口。
```

* @n 0x06 - 虚拟串口，带复合HID，此时HID的VID=0471，PID=7003。

* @n 0x07 - 表示控制发送和中断接收，此时VID=0471，PID=7002，支持磁条卡自动上传功能。

*/

```
public static native String  
dc_SetUsbMode(int ucmode);
```

```
public static native String  
dc_settime_hex(String time);
```

```
/**
```

* @return 成功: "0000|卡序列号"

* 失败: "错误码|错误提示信息" {@link
com.deccard.driver.references.ErrorCode}

* @brief 寻FM11RF005卡。

* @par 说明:

* 对FM11RF005卡进行寻卡操作。

* @param[in] _Mode 模式，模式，同
::dc_request 的 @a _Mode 。

*/

```
public static native String  
dc_card_fm11rf005(int _Mode);
```

```

/**
    * @return 成功: "0000|(读取的值)"
<br>
    * 失败: "错误码|错误提示信息" {@link
com.decard.driver.references.ErrorCode}
    * @brief 读FM11RF005卡。
    * @par 说明:
    * 读取FM11RF005卡的数据。
    * @param[in] _Adr 块地址(0~15)。
    */
public static native String
dc_read_fm11rf005(int _Adr);

/**
    * 获取ARQC
    *
    * @return 卡号|应用主序列号|持卡人名称|身份证号|有效日期|二磁道数据|ARQC;
    * @param[in] cardType 操作类型
    * @n 1 - 接触式
    * @n 2 - 非接触式
    * @param[in] txData 交易数据 (标签+3
位长度+内容,例
如"P0120000000000000Q012000000000000R003
156S00820120425T00296U006115139V0250000
000000000000000000000000")

```

```
    * @param[in] aidList 应用列表 (格式
    ASCII,默认为空,从卡片中自动获取)
    */
public static native String
dc_GetArqc(int cardType, String txData,
String aidList);

public static native String
dc_DfuAccess();

public static native String
dc_DfuBegin(int fileType, long
fileLen);

public static native String
dc_DfuPocess(int fileType, byte[]
fileData);

public static native String
dc_wordStockBegin(int fileType, long
fileLen);

public static native String
dc_wordStockPocess(int fileType, int
packageNum, long fileLen, byte[]
fileData);
```

```
public static native String
dc_dcrf32_json_parse(String jsonData);

public static native String
ReadHealthCardInfo();

/**
 * @return 成功: HealthCard 对象
<br>
 * 失败: null
 * @brief 读取健康卡信息
 */
public static HealthCard
ReadHealthCard();

/*****
*****
*****/
//根据已配对蓝牙设备名称连接蓝牙读卡器
public static int
dc_openBluetooth(String var0, Context
var1, String var2, int var3);

//获取非接触式三代卡卡识别码
public static String
dc_SSCard_Serial_Number();
```

```

/**
    * @param type 1:接触式 2:非接
    * @return 成功: "0000|卡号|二磁道数
    据" <br>
    * 失败: "错误码|错误提示信息" {@link
    com.decard.driver.references.ErrorCode}
    <br>
    */
public static String dc_readIcCard(int
type);
/**
    * 请求卡，在操作区返回卡的UID和DSFID
    * @param标记0:查找单个卡建议，1:查找多
    个卡建议
    * AFI应用程序家族标识符
    * @return 成功:0000|(DSFID(1字
    节)+UID(8字节))，失败:ffff|错误信息
    */
public static native String
dc_inventory_hex(int flags, int AFI);

/**
    * 使指定的15693卡进入静默状态。
    * @param[in] UID 卡片UID，固定为8个
    字节。
    * @return 成功:0000|，失败:ffff|错误
    信息

```



```
    */  
public static native String  
dc_stay_quiet_hex(String UID);  
  
/**  
 * 使指定的15693卡进入选择状态。  
 * @param[in] UID 卡片UID，固定为8个字  
节。  
 * @return 成功:0000|，失败:ffff|错误  
信息  
 */  
public static native String  
dc_select_uid_hex(String UID);  
  
/**  
 * 使指定的15693卡进入准备状态。  
 * @param[in] UID 卡片UID，固定为8个字  
节。  
 * @return 成功:0000|，失败:ffff|错误  
信息  
 */  
public static native String  
dc_reset_to_ready_hex(String UID);  
  
/**  
 * @brief 15693卡读块数据。
```

```

        * @param startblock起始块地址，取值范围0-27或0-63
        * 可以读取一次的块号，范围1-10
        * @param[in] UID 卡片UID，固定为8个字节。
        * @return 成功:0000|数据缓冲区内部的块,失败: fail:ffff|错误信息
    */
public static native String
dc_readblock_hex(int startblock, int
blocknum, String UID);

/**
    * @brief 15693卡写块数据。
    * @param startblock起始块地址，取值范围0-27或0-63
    * 可以读取一次的块号，范围1-10
    * @param[in] UID 卡片UID，固定为8个字节。
    * @return 成功:0000|写入内部块的数据缓冲器,失败:ffff|错误信息
    */
public static native String
dc_writeblock_hex(int startblock, int
blocknum,String UID,String data);

/**

```

```

    * 15693卡锁块。
    * @param 块起始块地址，取值范围0-27或0-
63
    * @param[in] UID 卡片UID，固定为8个字
节。
    * @return 成功:0000|，fail:失败|错误
信息
    */
    public static native String
dc_lock_block_hex (int block, String
UID);

    /**
    * 15693卡写AFI。
    * @param[in] AFI 应用族识别符。
    * @param[in] UID 卡片UID，固定为8个字
节。
    * @return 成功:0000|，失败:ffff|错误
信息
    */
    public static native String
dc_write_afi_hex(int AFI, String UID);

    /**
    * 15693卡锁AFI。
    * @param[in] AFI 应用族识别符。

```

```

        * @param[in] UID 卡片UID，固定为8个字
节。

        * @return 成功:0000|， 错误:ffff|错误
信息

        */
public static native String
dc_lock_afi_hex(int AFI, String UID);

    /**
        * @brief 15693卡写DSFID
        * @param[in] DSFID 数据存储格式识别
符。

        * @param[in] UID 卡片UID，固定为8个字
节。

        * @return 成功:0000|， 失败:ffff|错误
信息

        */
public static native String
dc_write_dsfid_hex(int DSFID, String
UID);

    /**
        * @brief 15693卡锁DSFID。
        * @param[in] DSFID 数据存储格式识别
符。

        * @param[in] UID 卡片UID，固定为8个字
节。

```

```

        * @return 成功:0000|, 失败:ffff|错误
信息
    */
public static native String
dc_lock_dsfid_hex(int DSFID, String
UID);

/**
 * @brief 获取15693卡系统信息
 * @param[in] UID 卡片UID, 固定为8个字
节。
 * @return 成功:0000|, 失败:ffff|错误
信息
    */
public static native String
dc_get_systeminfo_hex(String UID);

/**
 * @brief 获取15693卡安全信息。
 * @param[in] startblock 起始块地址,
从0x00开始编号。
 * @param[in] blocknum 块数量, 范围
1~10。
 * @param[in] UID 卡片UID, 固定为8个字
节。

```

```
        * @return 成功:0000|, 失败:ffff|错误
        信息
    */
    public static native String
    dc_get_securityinfo_hex(int startblock,
    int blocknum, String UID);

    /**
        * @return 成功: 0000|返回的UID|返回
        的复位信息。 失败:ffff|错误信息
        * @brief Felica卡复位。
    */
    public static native String
    dc_FelicaReset();

    /**
        * @return 成功: 0000|返回的数据。 失
        败:ffff|错误信息
        * @brief Felica卡指令交互。
        * @param sdata 发送的数据。
    */
    public static native String
    dc_FelicaApdu(String sdata);

    /**
        * @brief 获取身份证卡片种类。
```

```

    * @par      说明：
    * 获取身份证卡片种类。
    * @param[in] info_len 文字信息的长
度。

    * @param[byte[]] info 文字信息。
    * @return <0表示失败，==0表示中国人居
民身份证，==1表示外国人永久居留证（2017版），
==2表示港澳台居民居住证，==3表示外国人永久居留
证（新版）。

    */
    public static native short
dc_GetIdCardType(int info_len,byte[]
info);
/**

    * 继电器接口
    * @param lednum 传4为控制闸机，其他控
制灯光颜色：0:"red"    1:"green"
2:"white"    3:"ir"
    * @param status 0关闭 1打开
    */
    public static native void
led_control(int lednum,int status);

```

